B.Tech. BCSE497J - Project-I

CARDIOSENTINEL: A MULTI-MODEL FEDERATED FRAMEWORK FOR ECG-BASED HEART DISEASE DETECTION

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Programme

by

23BDS0210 SANVI SAKET

23BDS0243 SIDDHI SALUNKHE

Under the Supervision of

<Project Guide Name <Bold>>

<Designation>

School of Computer Science and Engineering (SCOPE)



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

DECLARATION

We hereby declare that the project entitled “CardioSentinel: A Multi-Model Federated Framework for ECG-Based Heart Disease Detection” submitted by us, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering* to VIT is a record of bonafide work carried out by us under the supervision of Prof. / Dr. << Guide Name >>.

We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Vellore

Date :

Signature of the Candidates

CERTIFICATE

This is to certify that the project entitled “CardioSentinel: A Multi-Model Federated Framework for ECG-Based Heart Disease Detection” submitted by **Sanvi Saket (23BDS0210) and Siddhi Salunkhe (23BDS0243)**, **School of Computer Science and Engineering**, VIT, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering*, is a record of bonafide work carried out by them under my supervision during Fall Semester 2026-2027, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place : Vellore

Date :

Signature of the Guide

Examiner(s)

<< Head of the Department >>

<< Programme >>

ACKNOWLEDGEMENTS

We are deeply grateful to the management of Vellore Institute of Technology (VIT) for providing us with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in our academic journey. The support and infrastructure provided by VIT have enabled us to explore and develop our ideas to their fullest potential.

Our sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering (SCOPE), for his unwavering support and encouragement. His leadership and vision have greatly inspired us to strive for excellence. The Dean's dedication to academic excellence and innovation has been a constant source of motivation for us. We appreciate his efforts in creating an environment that nurtures creativity and critical thinking.

We express our profound appreciation to <Head of Department's Name>, the Head of the <Department Name>, for his/her insightful guidance and continuous support. His/her expertise and advice have been crucial in shaping the direction of our project. The Head of Department's commitment to fostering a collaborative and supportive atmosphere has greatly enhanced our learning experience. His/her constructive feedback and encouragement have been invaluable in overcoming challenges and achieving our project goals.

We are immensely thankful to our project guide, <Guide's Name>, for his/her dedicated mentorship and invaluable feedback. His/her patience, knowledge, and encouragement have been pivotal in the successful completion of this project. Our supervisor's willingness to share his/her expertise and provide thoughtful guidance has been instrumental in refining our ideas and methodologies. His/her support has not only contributed to the success of this project but has also enriched our overall academic experience.

Thank you all for your contributions and support.

Sanvi Saket
Siddhi Salunkhe

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	**xii**
1.	**INTRODUCTION**	**1**
	1.1 Background	**1**
	1.2 Motivations	**1**
	1.3 Scope of the Project	**2**
2.	**PROJECT DESCRIPTION AND GOALS**	**3**
	2.1 Literature Review	**3**
	2.1.1 Unimodal Deep-Learning Approaches	**3**
	2.1.2 Multimodal Approaches	**3**
	2.1.3 Federated-Learning Approaches	**3**
	2.1.4 Explainable AI Approaches	**4**
	2.2 Research Gap	**5**
	2.3 Objectives	**5**
	2.4 Problem Statement	**5**
	2.5 Project Plan	**6**
3.	**TECHNICAL SPECIFICATION**	**7**
	3.1 Requirements	**7**
	3.1.1 Functional	**7**
	3.1.2 Non-Functional	**7**
	3.2 Feasibility Study	**8**
	3.2.1 Technical Feasibility	**8**
	3.2.2 Economic Feasibility	**8**
	3.2.3 Social Feasibility	**8**
	3.3 System Specification	**8**
	3.3.1 Hardware Specification	**8**
	3.3.2 Software Specification	**8**
4.	**DESIGN APPROACH AND DETAILS**	**10**
	4.1 System Architecture	**10**
	4.2 Design	**11**
	4.2.1 Data Flow Diagram	**11**
	4.2.2 Use Case Diagram	**12**
	4.2.3 Class Diagram	**13**
	4.2.4 Sequence Diagram	**14**

5.	**METHODOLOGY AND TESTING**	**16**
	5.1 Module Description	**16**
	5.1.1 Data Preparation	**16**
	5.1.2 Pre-processing and Augmentation	**16**
	5.1.3 Client Models	**17**
	5.1.4 Local Training and Calibration	**17**
	5.1.5 Federated Aggregation	**18**
	5.1.6 Per-Finding Triage	**19**
	5.1.7 Explainability	**19**
	5.1.8 Evaluation Metrics	**19**
	5.1.9 Printed-ECG Reading (Photos, Scans and PDFs)	**20**
	5.1.10 Web Portal and Voice Assistant	**20**
	5.2 Testing	**20**
	5.2.1 Automated Tests	**20**
	5.2.2 Protocol and Leakage Checks	**21**
	5.2.3 Upload Smoke Test	**21**
6.	**PROJECT DEMONSTRATION**	**22**
7.	**RESULT AND DISCUSSION**	**27**
	7.1 Dataset	**27**
	7.2 Comparison with Traditional Machine Learning and Single Clients	**27**
	7.3 Ablation Experiments	**30**
	7.4 Federated Learning Analysis	**31**
	7.5 Class-wise Analysis and Triage	**33**
	7.6 Explainability Results	**34**
	7.7 Reading Photos of Paper ECGs	**36**
	7.8 Comparison with the State of the Art	**36**
	7.9 Cost Analysis	**37**
8.	**CONCLUSION**	**38**
9.	**REFERENCES**	**39**
	APPENDIX A – SAMPLE CODE	**43**

List of Figures

Figure No.	Title	Page No.
2.1	Project timeline	6
4.1	Architecture of the proposed CardioSentinel model	10
4.2	Federated workflow	11
4.3	Data flow diagram, level 0	12
4.4	Data flow diagram, level 1	12
4.5	Use case diagram of the CardioSentinel portal	13
4.6	Class diagram	14
4.7	Sequence diagram of a patient heart check	15
5.1	Algorithm of the complete CardioSentinel method	16
5.2	One-shot federated protocol and gate selection	19
6.1	Welcome page of the CardioSentinel portal	22
6.2	Patient home	23
6.3	'Check my heart'	23
6.4	Result page	24
6.5	Voice assistant that answers from the patient's own records	24
6.6	Doctor workspace	25
6.7	12-lead ECG viewer	25
6.8	Patient shares inbox in the doctor workspace	26
7.1	Number of records carrying each diagnostic superclass in the training, validation and test…	27
7.2	Test macro AUROC of the four clients and the three aggregation variants	29
7.3	Static aggregation weights learned on fold 9 for each seed	31
7.4	Local training of the four clients	32
7.5	Macro AUROC of the global	32

7.6	Convergence of the parameter-averaging frameworks	33
7.7	Per-class confusion matrices of the proposed model on the test fold	33
7.8	ROC curves of the proposed model on the test fold	34
7.9	Global explanation	35
7.10	Local explanations for the most confident true-positive MI and STTC validation records	35

List of Tables

Table No.	Title	Page No.
2.1	Summary of related work	4
2.2	Project milestones	6
3.1	Functional requirements	7
3.2	Non-functional requirements	7
3.3	Hardware specification	8
3.4	Software specification	8
5.1	Pre-processing steps and parameters	17
5.2	Learned features per client	17
5.3	Hyperparameters	18
5.4	Automated test suite	20
7.1	Dataset details	27
7.2	Comparison with traditional machine-learning models, the individual deep clients and the…	28
7.3	Traditional classifiers trained locally	29
7.4	Single clients versus the federated gate	29
7.5	Client-subset ablation	30
7.6	Shapley value φi of each client for the macro AUROC of the equal-weight federation	30
7.7	Ablation of the fusion strategy	30
7.8	Activation-function ablation	31
7.9	Optimizer ablation	31
7.10	Client-wise training summary	32
7.11	Comparison with federated-learning frameworks on the same four client partitions	33
7.12	Class-wise performance of the proposed model on the test fold	34
7.13	Deletion test	35
7.14	Real fold-10 photos	36
7.15	Comparison with state-of-the-art results on the PTB-XL five-superclass task	36
7.16	Cost analysis	37

List of Abbreviations

1D	One-Dimensional
AdamW	Adam with Decoupled Weight Decay
AI	Artificial Intelligence
API	Application Programming Interface
AUPRC	Area Under the Precision–Recall Curve
AUROC	Area Under the Receiver Operating Characteristic Curve
BCE	Binary Cross-Entropy
BiLSTM	Bidirectional Long Short-Term Memory
CD	Conduction Disturbance
CI	Confidence Interval
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
CV	Cross-Validation
DFD	Data Flow Diagram
ECG	Electrocardiogram
F1	Harmonic Mean of Precision and Recall
FedAvg	Federated Averaging
FedNova	Federated Normalized Averaging
FedProx	Federated Proximal Optimization
FL	Federated Learning
HYP	Hypertrophy
IG	Integrated Gradients
KNN	K-Nearest Neighbours
LLM	Large Language Model
LSTM	Long Short-Term Memory
MI	Myocardial Infarction
MLP	Multi-Layer Perceptron
MPS	Metal Performance Shaders

NORM	Normal ECG
NPV	Negative Predictive Value
NPY	NumPy Array File
PDF	Portable Document Format
PTB-XL	Physikalisch-Technische Bundesanstalt Extra-Large ECG Dataset
ReLU	Rectified Linear Unit
ResNet	Residual Network
ROC	Receiver Operating Characteristic
SCAFFOLD	Stochastic Controlled Averaging for Federated Learning
SCP-ECG	Standard Communications Protocol for Computer-Assisted Electrocardiography
SD	Standard Deviation
SNR	Signal-to-Noise Ratio
SPA	Single-Page Application
SQL	Structured Query Language
STTC	ST/T Change
SVM	Support Vector Machine
WFDB	Waveform Database (PhysioNet format)
XAI	Explainable Artificial Intelligence
XGBoost	Extreme Gradient Boosting

Symbols and Notations

x	Input ECG, 12 leads × 1,000 samples (10 s at 100 Hz)
yc	Label of superclass c ∈ {NORM, MI, STTC, CD, HYP} (multi-hot)
K	Number of clients (K = 4: A, B, C, D)
Pi(y | x)	Calibrated class probabilities of client i
P(y | x)	Global (server) probabilities after aggregation
wi	Aggregation weight of client i, wi ≥ 0, Σ wi = 1
Ti	Temperature of client i (fitted on fold 9)
zic	Logit of client i for class c
σ(·)	Logistic sigmoid
βc	Positive-class weight Nc^{-} / Nc^{+} in the loss
τc	Decision threshold of class c (maximises F1 on fold 9)
lc, hc	Rule-out and rule-in thresholds of the triage band of class c
Hi	Mean binary entropy of client i (input of the dynamic gate)
IGℓt	Integrated-Gradients attribution of lead ℓ at time t
φi	Shapley value of client i for the macro AUROC
v(S)	Macro AUROC of the equal-weight sub-federation S

ABSTRACT

Cardiovascular diseases are the leading cause of death worldwide, and the 12-lead electrocardiogram (ECG) is the first, cheapest and most widely available test for myocardial infarction, ischaemic ST/T changes, conduction disturbances and ventricular hypertrophy. Deep networks can read ECGs at a high level, but they are trained on data pooled at one site, whereas hospitals cannot share patient records. Federated learning removes the need to pool data, yet the common methods average model parameters over many communication rounds and therefore force every hospital to train the same network.

This project builds CardioSentinel, a one-shot federated system for multi-label ECG classification. Four clients, standing for four hospitals, each train a structurally different network on their own, patient-separated share of the PTB-XL dataset: a 1D CNN, a 1D ResNet, a bidirectional LSTM and a CNN-LSTM. Each client calibrates its output with a single temperature and shares only five class probabilities per ECG. A server combines them with weights on the probability simplex. Equal weights, learned static weights and a per-patient dynamic gate are compared with patient-grouped cross-validation on the validation fold, and a more complex combiner is adopted only if it improves the validation macro AUROC by at least 0.002.

On 21,388 PTB-XL records (80% training, 10% validation, 10% held-out test, three random seeds), the selected equal-weight gate reaches a test macro AUROC of 0.905, a macro F1 of 70.7% and a label-wise accuracy of 86.9%. It improves on the best single client in every seed (+0.009 to +0.012 AUROC, paired bootstrap) and comes within 0.004 AUROC of a model trained on all data pooled together and within 0.002 of FedAvg, which needs 20 communication rounds and a shared architecture. Integrated Gradients show which leads and waveform segments drove each finding, and Shapley values show how much each hospital contributes. A per-finding triage rule reports 53.7% of findings automatically at a precision of 0.937 and an accuracy of 0.967, and refers the rest to a cardiologist.

The model is deployed in a clinician and patient web portal. Patients can upload a digital ECG or a photo, scan or PDF of a paper ECG, read a plain-language result, keep a symptom journal and decide what their doctor sees; clinicians review a queue of AI pre-reads, view the 12-lead tracing and sign off results. Photos are digitized with a U-Net trace segmenter; on 100 real fold-10 photos the photo pipeline reaches a macro AUROC of 0.841, which is below the triage target, so every photo finding is referred to a doctor. The project shows that most of the benefit of pooling ECG data can be obtained without exchanging data, parameters or architectures.

CHAPTER 1

INTRODUCTION

1.1 Background

Cardiovascular diseases remain the leading cause of death worldwide, and the Global Burden of Disease study estimated that their prevalence almost doubled between 1990 and 2019 [1]. The 12-lead electrocardiogram (ECG) is the first-line, low-cost and non-invasive test for most of these conditions. It is recorded in primary care, emergency departments and wards, and a single ten-second recording can reveal myocardial infarction (MI), ischaemic ST/T changes (STTC), conduction disturbances (CD) and ventricular hypertrophy (HYP). Because a large fraction of ECGs is acquired where no cardiologist is present, reliable automatic interpretation directly affects how quickly a patient with an acute finding is identified and referred.

In current practice the ECG is read visually by a physician, often supported by rule-based software built into the electrocardiograph. Visual reading depends on training and workload, and the built-in algorithms make frequent errors, especially for non-sinus rhythms and subtle morphological changes, so their statements have to be over-read by an expert [2]. A single ECG can also carry several co-existing diagnoses, which makes the task multi-label rather than a choice between mutually exclusive classes.

Deep learning has substantially changed automatic ECG analysis. Convolutional and recurrent networks have reached cardiologist-level performance on single-lead ambulatory recordings [3] and on large 12-lead cohorts [4], and the public PTB-XL dataset [5] has enabled reproducible benchmarks in which ResNet, Inception and LSTM models reach a macro area under the ROC curve (AUROC) of 0.921–0.930 on the five-superclass task [6]. Later work added self-supervised pre-training [7], [8], state space models and patient metadata [9], and ECG–text multimodal learning [10], [11]. Almost all of these models are, however, trained on data pooled at one site.

1.2 Motivations

ECG archives are held by separate hospitals that cannot share patient records. Federated learning (FL) [12], [13] lets institutions train together without moving data, but the usual methods – FedAvg [14], FedProx [15] and SCAFFOLD [16] – average model parameters. Every hospital must therefore train the same network and take part in many synchronised communication rounds. In practice, hospitals have different computing resources and in-house models, and repeated synchronisation is difficult to organise.

Clinical use also requires more than a high AUROC. A clinician needs to see why the model reached a finding, and the system must state explicitly when it should not decide on its own. Most ECG studies provide neither [17]. These three observations – data that cannot be pooled, institutions that want their own architectures, and the need for explanations and safe referral – motivate this project.

The project asks a deliberately conservative question: if each hospital trains its own network once and shares only calibrated class probabilities, is the combined model better than the

best single hospital, and how close does it come to a model trained on all data pooled together?

1.3 Scope of the Project

In scope. The project covers:

- multi-label classification of 10-s, 12-lead ECGs (100 Hz) into the five PTB-XL diagnostic superclasses NORM, MI, STTC, CD and HYP;
- four simulated institutions (clients) that hold disjoint, patient-separated shares of the PTB-XL training folds and train four different architectures;
- one-shot, prediction-level aggregation with equal, static and dynamic weights, selected on validation data;
- calibration, per-class thresholds and a per-finding triage rule that refers uncertain findings;
- explanations with Integrated Gradients (leads and time) and Shapley values (clients);
- comparison with traditional machine learning, a centrally trained model and FedAvg, FedProx, SCAFFOLD and FedNova;
- the CardioSentinel web portal for patients and clinicians, including reading photos, scans and PDFs of paper ECGs.

Out of scope. Deployment across real hospitals, formal privacy guarantees such as differential privacy, rhythm and form statements beyond the five superclasses, regression targets such as severity scores, and clinical validation with cardiologists. The portal is a research prototype and not a medical device.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

This section reviews peer-reviewed studies published between 2022 and 2026 that are closest to the project. They are grouped into unimodal deep-learning approaches for the ECG, multimodal approaches, federated-learning approaches and explainable approaches. The reported figures are those stated by the authors on their own datasets and splits, so they are not directly comparable with each other. Table 2.1 summarises the review.

2.1.1 Unimodal Deep-Learning Approaches

Mehari and Strodthoff (2022) [7] adapted contrastive predictive coding, SimCLR, BYOL and SwAV to 12-lead ECGs from PTB-XL and reached a macro AUROC of 94.18% on the 71-statement task after self-supervised pre-training and fine-tuning, but the gain over supervised training was about one percentage point and pre-training relied on pooling several large corpora at one site. Na et al. (2024) [8] proposed ST-MEM, a spatio-temporal masked auto-encoder, and reached 82.5% accuracy and a macro AUROC of 93.3% on the five superclasses, but the method needs large-scale centralised pre-training and does not explain its decisions. Mehari and Strodthoff (2023) [9] applied structured state space (S4) models to the raw signal and reached a macro AUROC of 94.17% on the 71-statement task, again on a single pooled dataset. These studies set strong single-site baselines, but each assumes that all records can be gathered in one place.

2.1.2 Multimodal Approaches

Mehari and Strodthoff (2023) [9] also fused demographic metadata with the S4 representation and reached 94.63% macro AUROC, only 0.18 percentage points more. Li et al. (2023) [10] proposed METS, which aligns ECGs with generated reports through a frozen language model and reached 84.2% zero-shot accuracy on the superclass task, but its macro F1 of 65.7% stayed below the 77.6% of a supervised ResNet-18. Liu et al. (2024) [11] proposed MERL, ECG –report pre-training with test-time clinical-knowledge prompts, and reached an average zero-shot AUROC of 75.2% on six datasets, but it needs a free-text report for every training ECG. Palakayala et al. (2025) [18] fused MRI, gait and speech for Parkinson's disease with hierarchical attention (94.2% accuracy), but with modalities from different cohorts and centralised training. Additional modalities can help, but every modality must be available for each patient at one site.

2.1.3 Federated-Learning Approaches

Raza et al. (2022) [19] combined federated transfer learning with an explanation module on MIT-BIH beats (98.9% accuracy on clean signals), but evaluated beats of a 48-record database rather than patient-level 12-lead diagnosis. Goto et al. (2022) [20] trained ECG and echocardiogram models across hospitals in the United States and Japan (AUROC 90%–96% internally, 93% externally), with the same architecture at every site and a single binary task. Quan et al. (2025) [21] proposed FedOcw, which optimises client-specific aggregation weights every round and outperformed FedAvg, FedProx, SCAFFOLD and FedNova (74.81%), but still averages parameters of one network over 100 rounds. Sathi et al. (2025) [22] proposed explainable FL on

ECG fiducial features with SHAP and LIME (92.3%), with one model family on all clients. In all of these studies the clients train the same model and communicate repeatedly.

2.1.4 Explainable AI Approaches

Jahmunah et al. (2022) [23] used DenseNet and CNN models with Grad-CAM on the PTB database to detect MI (accuracy above 95%), on a small cohort with qualitative heat maps. Wagner et al. (2024) [17] proposed building blocks for auditing ECG models on PTB-XL and found that saliency was the only attribution method that passed all their sanity checks, without proposing a new classifier. Pantelidis et al. (2025) [24] proposed ECG-XPLAIM, an inception-style 1D CNN with Grad-CAM trained on MIMIC-IV and validated on PTB-XL (98% macro AUROC for conduction disturbances), but its long-QT sensitivity was only 69.1%. Explainability is increasingly expected, but it is rarely combined with privacy-preserving training or with a rule that decides which predictions should be referred.

Table 2.1 Summary of related work (reported figures are the authors' own; metric named when not accuracy)

Ref	Author & Year	Dataset / Modality	Model / Method	Accuracy (%)	Limitation
[7]	Mehari & Strodthoff, 2022	PTB-XL; 12-lead ECG	CPC self-supervised pretraining + LSTM	94.18 (macro AUROC)	Centralised pretraining on pooled corpora
[8]	Na *et al.*, 2024	PTB-XL superclasses; ECG	ST-MEM masked auto-encoder	82.5 (AUROC 93.3)	Large centralised pretraining; no explanations
[9]	Mehari & Strodthoff, 2023	PTB-XL; ECG (+ metadata)	S4 state space model	94.17 / 94.63 (macro AUROC)	Single site; metadata adds 0.18 points
[10]	Li *et al.*, 2023	PTB-XL; ECG + report text	METS, frozen language model	84.2 (zero-shot)	F1 65.7 below supervised ResNet-18
[11]	Liu *et al.*, 2024	MIMIC-ECG + 6 datasets; ECG + text	MERL multimodal pretraining	75.2 (avg. zero-shot AUROC)	Needs paired reports; below fine-tuning
[18]	Palakayala *et al.*, 2025	PPMI, PhysioNet, voice; MRI, gait, speech	HAMF hierarchical attention fusion	94.2	Modalities from different cohorts; centralised
[19]	Raza *et al.*, 2022	MIT-BIH; single-lead beats	Federated transfer learning + XAI	98.9 (clean), 94.5 (noisy)	Beat-level task; same model on all clients
[20]	Goto *et al.*, 2022	4 hospitals; ECG, echo	Multinational FL	90–96 (AUROC)	Binary task; homogeneous architectures
[21]	Quan *et al.*, 2025	5 speech corpora	FedOcw optimised client weights	74.81	100 rounds of parameter averaging
[22]	Sathi *et al.*, 2025	MIT-BIH, ST-T, Fantasia; fiducial features	Explainable FL with SHAP/LIME	92.3	Hand-crafted features; inconsistent reporting
[23]	Jahmunah *et al.*, 2022	PTB; 12-lead ECG	DenseNet/CNN + Grad-CAM	>95	Small cohort; qualitative heat maps
[17]	Wagner *et al.*, 2024	PTB-XL; 12-lead ECG	Attribution sanity checks, concept analysis	– (not reported)	Explanation study, no new classifier
[24]	Pantelidis *et al.*, 2025	MIMIC-IV → PTB-XL; ECG	ECG-XPLAIM 1D CNN + Grad-CAM	98 (macro AUROC, CD)	Selected arrhythmias; LQT sensitivity 69.1

Ref	Author & Year	Dataset / Modality	Model / Method	Accuracy (%)	Limitation
This work	–	PTB-XL superclasses; 12-lead ECG	One-shot federated gate over 4 heterogeneous clients	86.9 (label-wise); AUROC 90.5	See Section 7.8

2.2 Research Gap

The review shows three gaps:

1. **Pooling.** Single-site ECG models reach a high AUROC, but they assume that all records can be pooled.
2. **Homogeneity and rounds.** Federated methods remove that assumption, but they average parameters, so every institution must adopt the same architecture and take part in many synchronised rounds.
3. **Explanation and referral.** Explainable models are seldom combined with privacy-preserving training or with an explicit rule for deferring uncertain findings to a clinician.

CardioSentinel addresses these gaps by letting each institution train its own architecture once, combining only calibrated probabilities through a validated simplex gate, explaining the combined prediction on the raw signal, and reporting automatically only those findings for which the validated precision and recall reach the clinical target.

2.3 Objectives

1. Build four heterogeneous ECG classifiers (1D CNN, 1D ResNet, BiLSTM, CNN-LSTM), each trained only on its own patient-separated share of PTB-XL.
2. Combine their calibrated probabilities with equal, static and per-patient dynamic weights on the probability simplex, and select the combiner on validation data with a parsimony rule fixed in advance.
3. Test, once and on held-out data over three seeds, whether the federated model beats the best single client, and measure how close it comes to centralised training and to FedAvg, FedProx, SCAFFOLD and FedNova.
4. Quantify the contribution of each client with subset ablations and Shapley values, and explain predictions with Integrated Gradients.
5. Design a per-finding triage rule that reports findings automatically only at precision, recall and accuracy ≥ 0.90 and refers the rest.
6. Deploy the model in a patient and clinician portal that also reads photos, scans and PDFs of paper ECGs.

2.4 Problem Statement

Given a 10-s, 12-lead ECG $x \in \mathbb{R}^{12\times1000}$, predict the multi-hot label vector $y \in \{0, 1\}^5$ over NORM, MI, STTC, CD and HYP. The training data are split among $K = 4$ institutions that may not exchange records or model parameters and that may each use a different network. The server may only receive each client's predicted class probabilities $P_i(y \mid x)$. The task is to find weights $w_i \geq 0$ with $\Sigma_i w_i = 1$ such that $P(y \mid x) = \Sigma_i w_i P_i(y \mid x)$ is better than the best individual client on

unseen patients, with every modelling choice made without looking at the test data, and to decide for each finding whether it can be reported automatically or must be referred.

2.5 Project Plan

The work was carried out in October 2026. Figure 2.1 shows the timeline and Table 2.2 the milestones, taken from the dated entries of the project log (RESULTS.md). The first week built an earlier three-modality prototype (ECG, clinical text and images, with other datasets); it was replaced on 15 October by the four-client design described in this report, and its code was removed on 17 October.

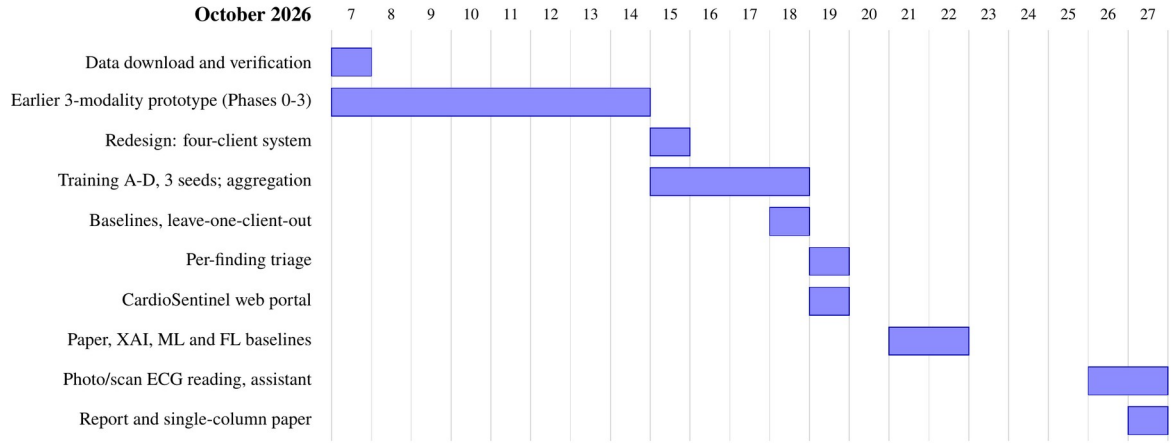


Figure 2.1 Project timeline (October 2026), from the dated entries of the project log

Table 2.2 Project milestones

Date (2026)	Milestone
7 Oct	PTB-XL downloaded and verified (21,799 records, 18,869 patients, folds 1–10)
7–14 Oct	Earlier three-modality prototype (Phases 0–3); replaced on 15 Oct
15 Oct	Redesign approved: four clients with different architectures and a weighted gate
18 Oct	Clients A–D trained over seeds 42–44; selection rule (≥ 0.002 margin) approved before testing; all-data baseline and leave-one-client-out
19 Oct	Per-finding triage (rule-in / rule-out) approved before using fold 10; CardioSentinel web portal (FastAPI + single-page app)
21–22 Oct	Paper draft, Integrated Gradients, traditional ML, FL baselines, ablations
26–27 Oct	Photo / scan / PDF reading, voice assistant, 3-D heart, portal redesign
27 Oct	Single-column paper, Word version and this report

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional

Table 3.1 Functional requirements

ID	Requirement
FR1	Prepare PTB-XL v1.0.3: map SCP codes to the five superclasses, drop unlabelled records, band-pass filter and store the signals (python -m cardiomoe.data.ptbxl --prepare).
FR2	Train four clients on their own folds (A 1–2, B 3–4, C 5–6, D 7–8) with early stopping on fold 9.
FR3	Fit one temperature per client and per-class decision thresholds on fold 9.
FR4	Fit equal, static and dynamic combiners by patient-grouped 5-fold CV on fold 9 and select one with the ≥ 0.002 macro-AUROC rule.
FR5	Evaluate every client and combiner once on fold 10 (AUROC with 95% bootstrap CI, AUPRC, precision, recall, specificity, F1, accuracy, confusion matrices, paired bootstrap).
FR6	Fit per-class triage bands on fold 9 and report coverage, precision, recall and accuracy on fold 10.
FR7	Explain predictions with Integrated Gradients and compute Shapley values over client subsets.
FR8	Portal: sign-up and sign-in for patients and clinicians with separated roles.
FR9	Portal: analyse an uploaded ECG (CSV, NPY, WFDB) or up to four photos / scans / PDFs of one paper ECG and show the verdict, the five findings and the rhythm strip.
FR10	Portal: symptom journal, sharing with the doctor (nothing is shared until the patient presses Send), messages, appointments and a voice assistant that answers from the patient's own records.
FR11	Portal: clinician queue of ECGs to review, 12-lead viewer, sign-off and release, replies to shares.

3.1.2 Non-Functional

Table 3.2 Non-functional requirements

ID	Requirement
NFR1 Privacy	Clients never share data or parameters; the server sees only five probabilities per client and ECG. Patient self-checks stay private until shared.
NFR2 Validity	Patient-level splits; fold 10 is used once (evaluate.py refuses a second run without --rerun); nothing is tuned on it.
NFR3 Reproducibility	All randomness seeded (seed 42); final results over seeds 42, 43, 44; every script supports --debug (about 200 samples, CPU, a few minutes).
NFR4 Safety	Findings outside the validated triage bands, and all photo findings, are referred to a doctor; every screen states that the result is an AI pre-read and not a diagnosis.
NFR5 Efficiency	Training and inference on an 8-GB laptop without a discrete GPU.
NFR6 Usability	Plain-language results for patients; standard 25 mm/s, 10 mm/mV ECG grid for clinicians.
NFR7 Maintainability	Modular Python package (cardiomoe), configuration in YAML, 37 automated tests.

3.2 Feasibility Study

3.2.1 Technical Feasibility

The complete pipeline – data preparation, training of four clients over three seeds, aggregation, evaluation, triage, explanations and the portal – was run on an Apple M1 MacBook Air with 8 GB of unified memory using PyTorch on the Metal (MPS) backend. A training epoch took 26.8 s for Client A, 38.5 s for Client B, 65.4 s for Client C and 28.2 s for Client D. Integrated Gradients needed the CPU, because MPS ran out of memory on 8 GB, and took about 47 s per batch of 16 records. The models are small (0.44M–6.73M parameters), so inference in the portal runs on the same machine. The project is therefore technically feasible with commodity hardware.

3.2.2 Economic Feasibility

All data are open access: PTB-XL v1.0.3 (CC BY 4.0) from PhysioNet and the ECG-Image-Database (CC BY-ND 4.0) from Kaggle. All software is open source (Python, PyTorch, scikit-learn, FastAPI, SQLite, Tectonic). No paid cloud computing or GPU was used. The only optional running cost is the large language model behind the voice assistant (OpenRouter, pay per use); without a key the assistant falls back to offline answers. The licence cost of the project is therefore zero (Section 7.9).

3.2.3 Social Feasibility

Automatic ECG pre-reading can shorten the time to referral where no cardiologist is present, and the federated design lets hospitals collaborate without moving patient records. The portal is patient-first: patients decide what their doctor sees, uncertain findings are referred rather than guessed, and every result is labelled as an AI pre-read. The main social risks – over-reliance on the model and unequal performance across populations – are addressed by the triage rule, by the explicit referral messages and by stating the limits of a single-country dataset.

3.3 System Specification

3.3.1 Hardware Specification

Table 3.3 Hardware specification

Component	Specification used
Machine	Apple M1 MacBook Air
Memory	8 GB unified memory
Accelerator	Apple GPU via Metal Performance Shaders (MPS); CPU for Integrated Gradients
Storage	PTB-XL records100 0.55 GB; ECG-Image-Database fold 10 about 2.5 GB (all 977 D0 records about 22 GB)
Client device	Any modern browser (desktop or phone) for the portal

3.3.2 Software Specification

Table 3.4 Software specification

Category	Software
Operating system	macOS
Language	Python 3.11.17
Deep learning	PyTorch 2.14.1 (torch ≥ 2.2)
Data and ML	NumPy, pandas, SciPy, scikit-learn, XGBoost, PyArrow, wfdb ≥ 4.1

Category	Software
Images	PyMuPDF ≥ 2.0, OpenCV (headless) ≥ 5.0, Pillow
Plots	Matplotlib ≥ 3.8
Web	FastAPI ≥ 0.110, Uvicorn ≥ 0.29, python-multipart, httpx; SQLite; HTML, CSS and JavaScript SPA
Testing	pytest ≥ 8.0
Assistant (optional)	OpenRouter API (OPENROUTER_API_KEY), offline fallback
Documents	Tectonic (LaTeX), pandoc

CHAPTER 4

DESIGN APPROACH AND DETAILS

4.1 System Architecture

Figure 4.1 shows the architecture. The data block at the bottom holds the PTB-XL records, split by the official stratified folds into four client partitions, a validation fold held by the server and a test fold. The pre-processing block applies the same signal pipeline at every client. Each client learns its own features with its own architecture and ends in a classifier head (C) that outputs five logits, which are converted into calibrated probabilities with a per-client temperature. The federated server combines the four probability vectors with simplex weights into the global output, the classification block turns it into five binary decisions and triage decisions, and the explainability block attributes the output to leads and time.

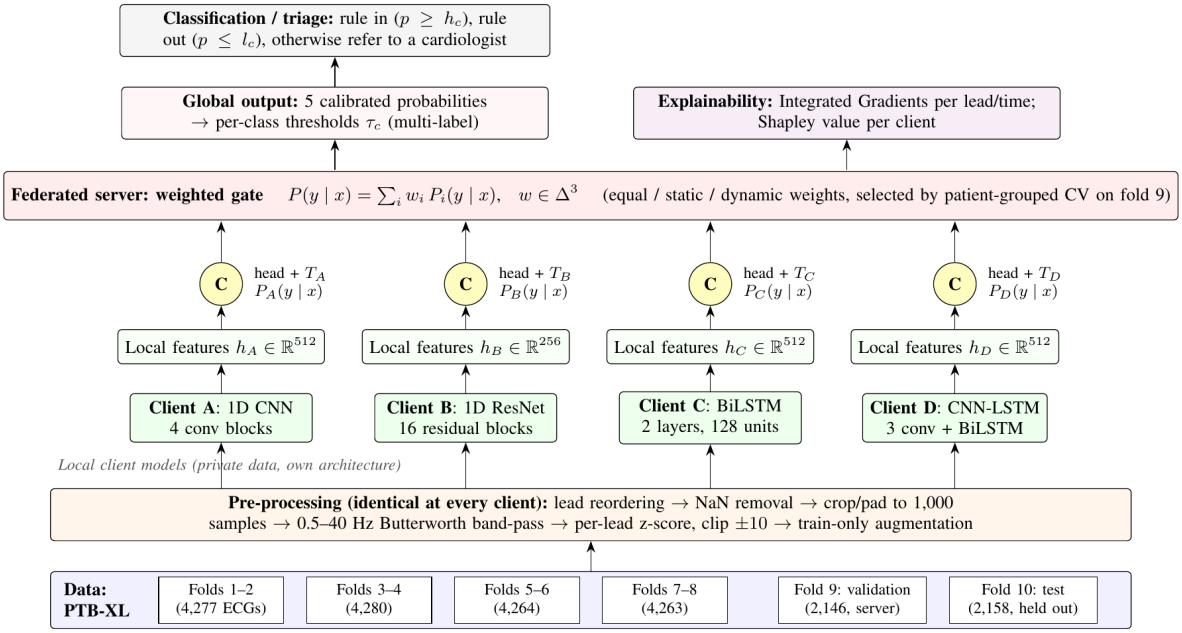


Figure 4.1 Architecture of the proposed CardioSentinel model

Figure 4.2 shows the federated workflow. Each client trains once on its private data and fits its temperature (step 1). Afterwards the server sends an ECG to the clients and receives calibrated probabilities (step 2), and mixes them (step 3). No parameters or raw training data are exchanged; at query time 20 numbers per ECG reach the server.

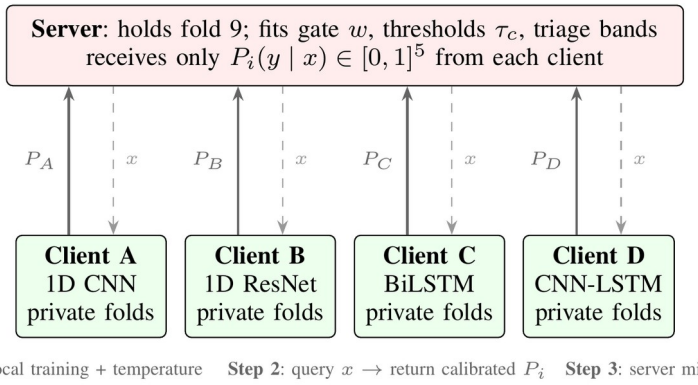


Figure 4.2 Federated workflow: clients train once on private data; the server queries them and receives only calibrated probabilities

The deployed system adds the CardioSentinel portal on top of this model. A FastAPI server exposes a REST API (for example POST /api/me/analyze for patients and POST /api/analyze for clinicians), stores users, ECGs, results, journal entries, messages and audit records in SQLite, and serves a single-page web application. The inference Engine loads the four seed-42 clients, the selected gate and the triage bands; photos, scans and PDFs are first converted to a 12 × 1,000 signal by the digitizer.

4.2 Design

4.2.1 Data Flow Diagram

Figure 4.3 is the context diagram (level 0). The system receives signals and labels from PhysioNet and ECG photos from Kaggle, receives ECG files, photos and symptoms from patients, and exchanges AI pre-reads and sign-offs with clinicians. Figure 4.4 refines it into five processes and three data stores.

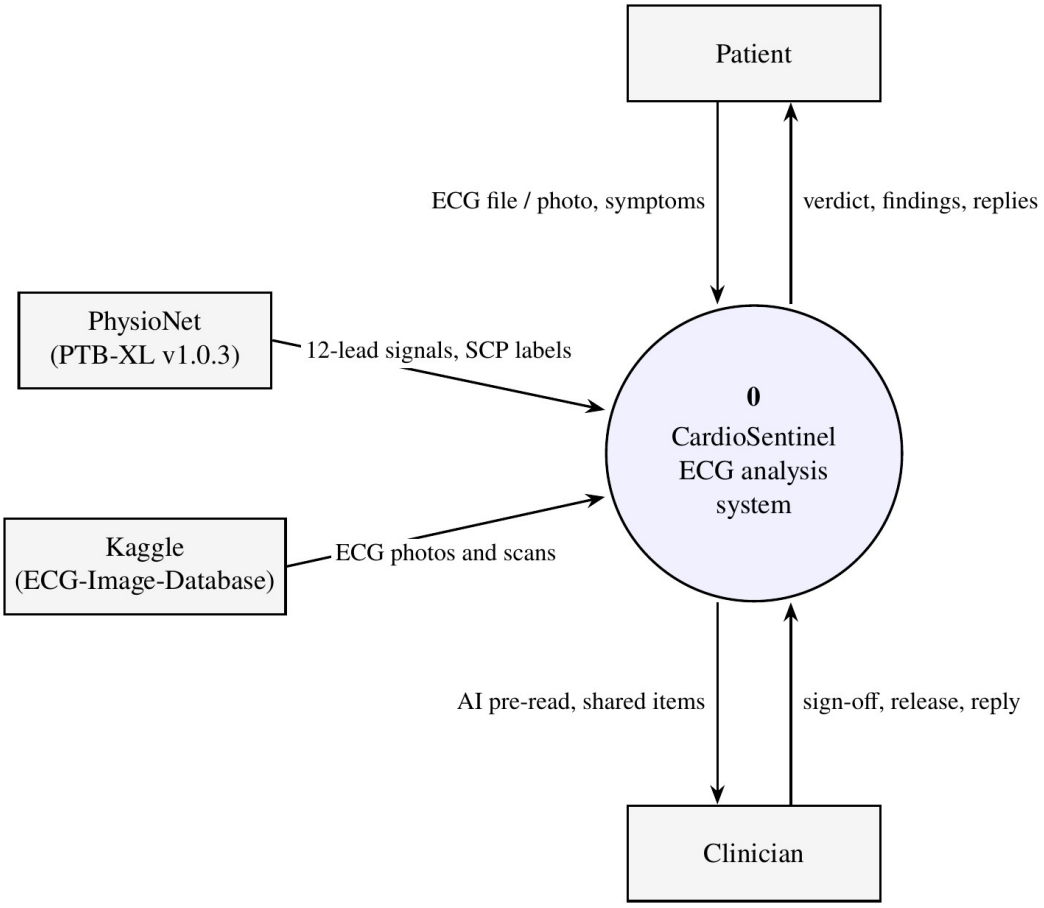


Figure 4.3 Data flow diagram, level 0 (context)

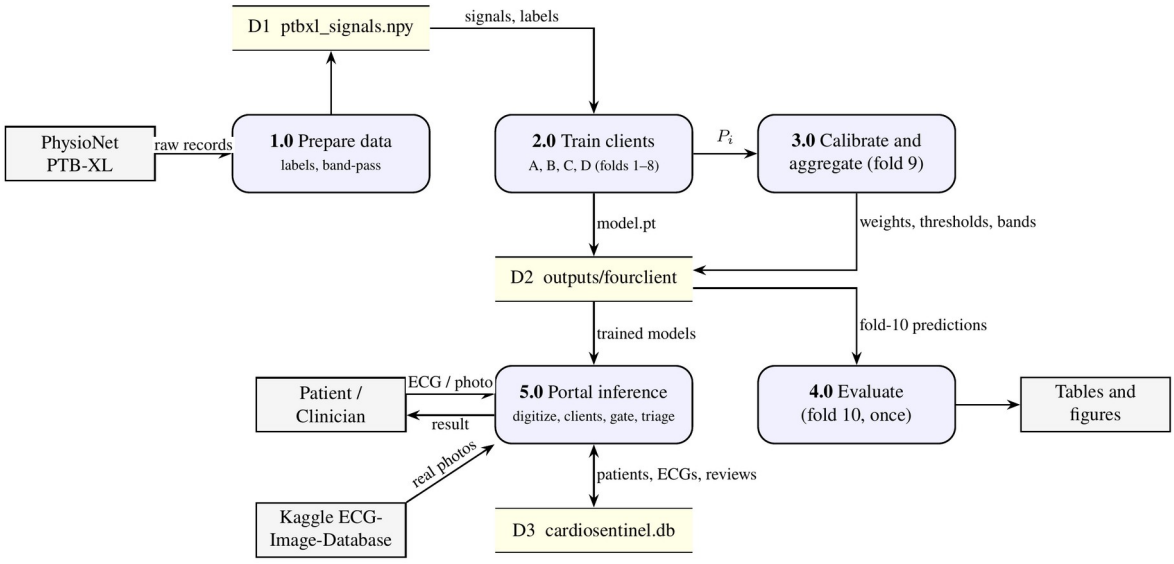


Figure 4.4 Data flow diagram, level 1

4.2.2 Use Case Diagram

Figure 4.5 shows the two actors of the portal. Patients sign up, check their heart, read results, keep a symptom journal, share with their doctor, ask the voice assistant and book appointments. Clinicians review the queue of ECGs, view the 12-lead tracing, sign off and release results, reply to shares and add ECGs for patients. Both analysis use cases include the AI pre-read.

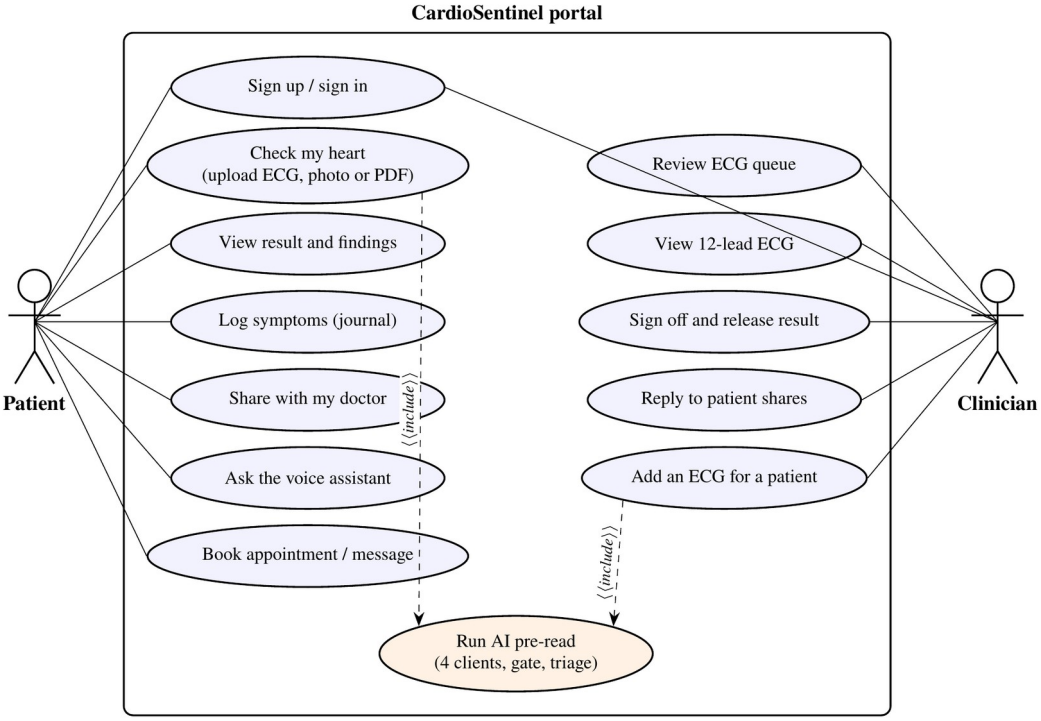


Figure 4.5 Use case diagram of the CardioSentinel portal

4.2.3 Class Diagram

Figure 4.6 shows the main classes. SignalClient (src/cardiomoe/fourclient/models.py) extends BaseClient, which holds the temperature buffer, and wraps one of four backbones: CNN1D, ResNet1D (made of ResBlock1d), BiLSTM or CNNLSTM. The three combiners in gate.py share the interface fit(), weights() and predict(); DynamicGate contains a GateNet MLP. The web Engine uses four SignalClients, the selected combiner and the Triage rule.

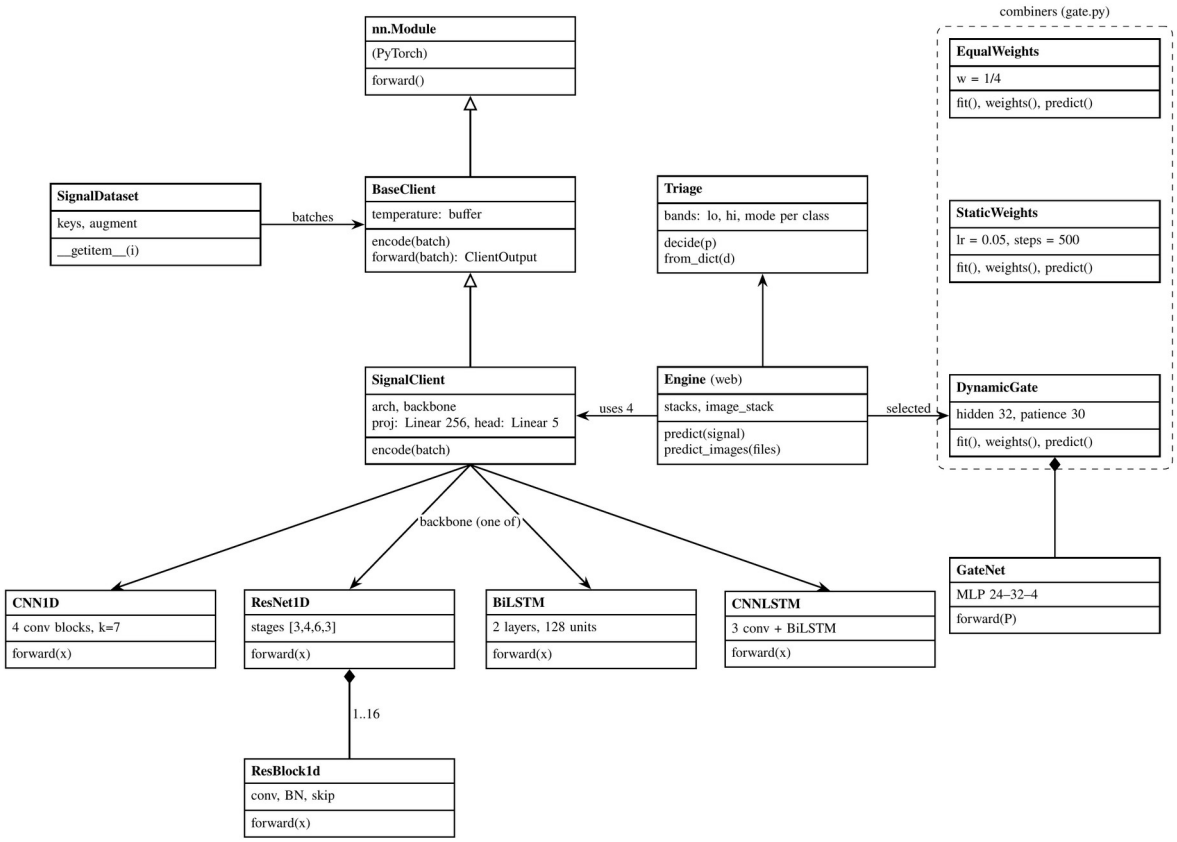


Figure 4.6 Class diagram (main classes of the model and the portal)

4.2.4 Sequence Diagram

Figure 4.7 shows a patient's heart check. The browser posts the file to /api/me/analyze. A photo, scan or PDF is first digitized; a CSV, NPY or WFDB file is parsed directly. The Engine runs the four clients, mixes their calibrated probabilities with the gate weights and applies the triage rule. The result is stored privately in SQLite and returned to the browser.

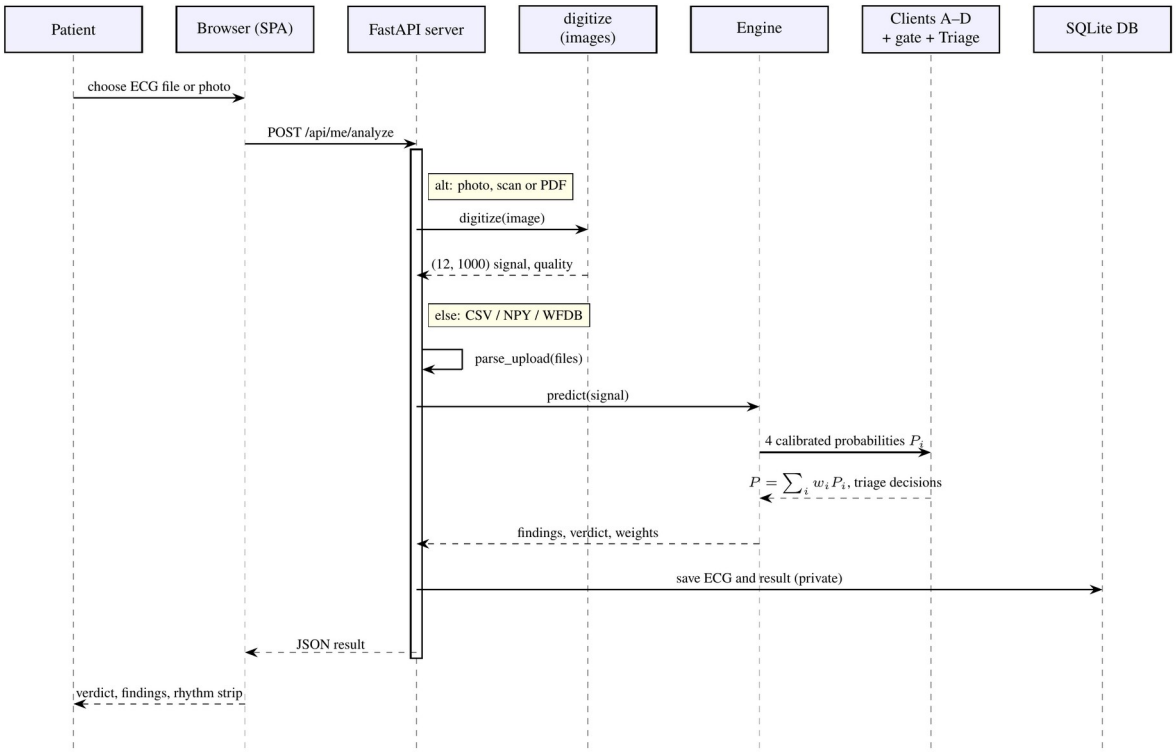


Figure 4.7 Sequence diagram of a patient heart check

CHAPTER 5

METHODOLOGY AND TESTING

Figure 5.1 summarises the complete method, from pre-processing to the explained, triaged prediction, and the following sections describe each module.

Algorithm 1 CardioSentinel: one-shot federated classification of 12-lead ECGs

Input: Client partitions $\mathcal{D}_A, \ldots, \mathcal{D}_D$ (folds 1–8), validation fold $\mathcal{V}$ (fold 9), test ECG x
Output: Calibrated probabilities $P(y \mid x)$, multi-label decision y^{pred}, triage status, attributions

for each client $i \in \{A, B, C, D\}$ **in parallel do** ▷ local, private
    Pre-process every record of $\mathcal{D}_i$ (band-pass, z-score, clip)
    Train network f_i with weighted BCE, AdamW and OneCycle; early-stop on macro AUROC of $\mathcal{V}$
    Fit temperature T_i on $\mathcal{V}$; $P_i(y \mid x) \leftarrow \sigma(f_i(x)/T_i)$
end for
Server collects $P_i(y \mid v)$ for all $v \in \mathcal{V}$ ▷ probabilities only
for combiner $g \in (\text{equal}, \text{static}, \text{dynamic})$ **do**
    Obtain out-of-fold predictions by patient-grouped 5-fold CV on $\mathcal{V}$; record macro AUROC a_g
end for
$g^\star \leftarrow$ equal; **for** g in (static, dynamic): **if** $a_g \geq a_{g^\star} + 0.002$ **then** $g^\star \leftarrow g$
Refit $g^\star$ on $\mathcal{V}$; tune per-class thresholds τ_c and triage bands (l_c, h_c) on its CV predictions
Inference: $P(y \mid x) \leftarrow \sum_i w_i^{g^\star}(x)\, P_i(y \mid x)$; $y_c^{\text{pred}} \leftarrow [P(y_c \mid x) \geq \tau_c]$
Triage: rule in if $P(y_c \mid x) \geq h_c$, rule out if $\leq l_c$, else refer
Explain $P(y_c \mid x)$ with Integrated Gradients over leads and time

Figure 5.1 Algorithm of the complete CardioSentinel method

5.1 Module Description

5.1.1 Data Preparation

bash scripts/download.sh downloads PTB-XL records100 (100 Hz) from PhysioNet's open-data mirror and writes a verified manifest (21,799 records, 18,869 patients). python -m cardiomoe.data.ptbxl --prepare maps each record's SCP-ECG statements to the diagnostic_class column of scp_statements.csv, keeping only the 44 statements flagged as diagnostic, and encodes them as a multi-hot vector over NORM, MI, STTC, CD and HYP. Records without a diagnostic superclass cannot be labelled and are removed (21,799 → 21,388); no other record is excluded. The official stratified folds are kept unchanged, so all recordings of a patient lie in one fold.

5.1.2 Pre-processing and Augmentation

Table 5.1 lists all pre-processing steps. They are implemented once and run identically by every client, so no client-specific statistics are needed and no information crosses client boundaries. The waveforms are read with WFDB, reordered to I, II, III, aVR, aVL, aVF, V1–V6, cleaned of non-finite values, cropped or padded to 1,000 samples and band-pass filtered (0.5–40 Hz, third-order Butterworth, forwards and backwards, so without phase shift). At load time each lead of each recording is z-scored with its own statistics and clipped:

$$x_{\ell t}^{\text{norm}} = \text{clip}\left(\frac{x_{\ell t} - \mu_\ell}{\sigma_\ell},\, -10,\, 10\right) \tag{5.1}$$

Because the statistics are per recording, normalisation needs no shared population statistics, which is essential in the federated setting. During training only, the signal is scaled by a random factor in [0.9, 1.1], Gaussian noise at an SNR of 20–30 dB and a 0.1–0.5 Hz baseline wander are added, and each lead is zeroed with probability 0.05.

Table 5.1 Pre-processing steps and parameters

Step	Method and parameters
Label encoding	SCP codes → 5 diagnostic superclasses (44 diagnostic statements); multi-hot
Record cleaning	Remove records without a superclass (21,799 → 21,388)
Lead order	I, II, III, aVR, aVL, aVF, V1–V6
Missing values	NaN/±∞ → 0
Length	100 Hz, crop/zero-pad to 1,000 samples (10 s)
Filtering	Butterworth band-pass 0.5–40 Hz, order 3, zero-phase
Normalisation	Per-lead, per-record z-score; clip to ±10; flat lead → 0
Augmentation (train)	Amplitude (0.9,1.1); noise SNR U(20,30) dB; wander 0.1–0.5 Hz, amp. U(0.05,0.3); lead dropout p=0.05
Class balancing	BCE positive weight =#neg/#pos per class

5.1.3 Client Models

The four clients span the main families of ECG networks, from purely convolutional to purely recurrent, so that their errors are as diverse as possible (Table 5.2). Client A is a plain 1D CNN with four blocks of a 7-tap convolution, batch normalisation, ReLU and max-pooling (32–256 channels). Client B is a 1D ResNet [25] with a 64-channel stem and stages of 64, 128, 256 and 256 channels with 3, 4, 6 and 3 residual blocks. Client C is a two-layer bidirectional LSTM [26], [27] with 128 hidden units per direction that reads all 1,000 time steps. Client D is a CNN-LSTM in which three convolutional blocks shorten the sequence to 125 steps before a one-layer BiLSTM. All clients summarise the time axis with concatenated average and max pooling and share the head Linear(d → 256) → ReLU → Dropout(0.3) → Linear(256 → 5).

Table 5.2 Learned features per client

Client	Feature extractor	Pooled dim.	Main signal property
A	4 conv blocks, k=7, 32–256 ch.	512	Local wave morphology
B	ResNet, 16 blocks, k=5, 64–256 ch.	512 → 256	Hierarchical morphology
C	2-layer BiLSTM, 128 units, 1,000 steps	512	Rhythm, beat-to-beat context
D	3 conv blocks + BiLSTM, 125 steps	512	Morphology and rhythm

5.1.4 Local Training and Calibration

Each client minimises the class-weighted binary cross-entropy

$$\mathcal{L}_i = -\frac{1}{5N}\sum_{n=1}^{N}\sum_{c=1}^{5}\left[\beta_c\, y_{nc}\log\sigma(z_{nc}) + (1-y_{nc})\log\left(1-\sigma(z_{nc})\right)\right] \tag{5.2}$$

with $\beta c = Nc^{-} / Nc^{+}$ computed from the client's own training labels, using AdamW (learning rate 10^{-3}, weight decay 10^{-2}), a OneCycle schedule, batch size 128, gradient clipping at 1.0 and up to 50 epochs with early stopping on the fold-9 macro AUROC (patience 8). After training, a single temperature Ti is fitted on fold 9 with LBFGS [28], which rescales the confidence without changing the ranking:

$$P_i(y_c \mid x) = \sigma(z_{ic}(x)/T_i) \tag{5.3}$$

Table 5.3 lists all hyperparameters; they are identical for every client so that only the

architecture and the data differ.

Table 5.3 Hyperparameters

Parameter	Value
Input	12×1000 (10 s at 100 Hz)
Number of clients	4 (A, B, C, D), 2 PTB-XL folds each
Communication rounds	1 (one-shot; probabilities only)
Local epochs	up to 50; early stopping, patience 8
Early-stopping criterion	macro AUROC on fold 9
Batch size	128
Optimizer	AdamW [39], weight decay 10^{-2}
Learning rate	10^{-3}, OneCycle schedule [40]
Loss	BCE with logits, positive weight N^−c/N⁺c
Gradient clipping	global norm 1.0
Head	Linear 256, ReLU, dropout 0.3, Linear 5
Activation	ReLU (BiLSTM: sigmoid/tanh gates)
Calibration	1 temperature per client, LBFGS on fold 9
Aggregation	equal / static / dynamic gate; selection by 5-fold patient-grouped CV, margin 0.002
Static gate	4 weights, Adam lr 0.05, 500 steps
Dynamic gate	MLP 24–32–4, dropout 0.2, AdamW lr 10^{-3}, ≤300 epochs, patience 30
Thresholds	per class, max. F1 on fold 9
Bootstrap	1,000 resamples (95% CI, paired)
Seeds	42, 43, 44

5.1.5 Federated Aggregation

The server aggregates the calibrated predictions,

$$P(y \mid x) = \sum_{i=1}^{K} w_i(x)\, P_i(y \mid x), \qquad w_i(x) \geq 0, \quad \sum_{i=1}^{K} w_i(x) = 1 \tag{5.4}$$

with the weights parameterised by a softmax so that the simplex constraint holds by construction. **Equal weights** use wi = 1/K. **Static weights** learn one logit per client by minimising the BCE of P(y | x) on validation data (Adam, learning rate 0.05, 500 steps). The **dynamic gate** is a per-patient mixture of experts [32]: an MLP takes the 20 client probabilities and the four mean binary entropies

$$H_i = -\frac{1}{5} \sum_{c} \left[p_{ic} \log p_{ic} + (1 - p_{ic}) \log(1 - p_{ic}) \right] \tag{5.5}$$

passes them through a 32-unit ReLU layer with dropout 0.2 and outputs w(x) by a softmax; its last layer starts at zero, i.e. from equal weights. All three are evaluated by patient-grouped five-fold cross-validation inside fold 9, and a more complex combiner is adopted only if its cross-validated macro AUROC exceeds that of the current choice by at least 0.002 (Figure 5.2). This rule was approved on 18 October 2026, before any three-seed test evaluation.

Algorithm 2 One-shot federated protocol and gate selection

Input: $K = 4$ clients with private data $\mathcal{D}_i$; server validation set $\mathcal{V}$ with patient IDs

Round 1 (only round): each client trains f_i on $\mathcal{D}_i$, fits T_i, exposes $P_i(y \mid x) = \sigma(f_i(x)/T_i)$

Server queries all clients on $\mathcal{V}$: $\mathbf{P} \in [0,1]^{|\mathcal{V}| \times K \times 5}$

Split $\mathcal{V}$ into 5 patient-grouped folds

for combiner g and CV fold j **do**

    Fit g on the other 4 folds by minimising BCE of $\sum_i w_i P_i$; predict fold j

end for

Select $g^\star$ by the ≥ 0.002 macro-AUROC rule; refit on all of $\mathcal{V}$

Deployment: for a new ECG, query clients and return $\sum_i w_i^{g^\star} P_i$

Figure 5.2 One-shot federated protocol and gate selection

5.1.6 Per-Finding Triage

When the clinical requirement is a precision and recall of at least 0.90, no single threshold can satisfy it for every class. For each class c a rule-out threshold lc and a rule-in threshold hc are therefore fitted on the selected combiner's cross-validated fold-9 predictions:

$$\hat{y}_c = \begin{cases} 1 \text{ (rule in)} & P(y_c \mid x) \geq h_c \\ 0 \text{ (rule out)} & P(y_c \mid x) \leq l_c \\ \text{refer to a cardiologist} & \text{otherwise} \end{cases} \tag{5.6}$$

Each band is the widest one with precision, recall and accuracy ≥ 0.92 (target + 0.02) and with at least 10% of the class's positives decided automatically (mode 'automated'). Otherwise the class is 'rule-out' only (negatives with NPV ≥ 0.95, ruling out ≤ 5% of positives) or 'review'. The frozen bands were applied once to the test fold.

5.1.7 Explainability

Because the equal-weight output Fc(x) = (1/K) Σi σ(zic(x)/Ti) is differentiable, it is explained with Integrated Gradients [33] with an all-zero baseline x′ and a 32-step Riemann sum:

$$\mathrm{IG}_{\ell t}(x) = (x_{\ell t} - x'_{\ell t}) \int_0^1 \frac{\partial F_c\big(x' + \alpha(x - x')\big)}{\partial x_{\ell t}}\, d\alpha \tag{5.7}$$

Global lead importance is the share of Σt |IGℓt| falling on each lead; a deletion test checks faithfulness by zeroing the most-attributed leads. At the federation level, the contribution of each client to the macro AUROC v(S) of an equal-weight sub-federation S is its Shapley value [36], computed exactly over the 15 non-empty subsets with v(∅) = 0.5:

$$\phi_i = \sum_{S \subseteq \mathcal{K} \setminus \{i\}} \frac{|S|!\,(K - |S| - 1)!}{K!} \big[v(S \cup \{i\}) - v(S) \big] \tag{5.8}$$

5.1.8 Evaluation Metrics

Every metric is computed per class from the confusion counts TPc, FPc, FNc and TNc on the N test records and then macro-averaged over the five classes:

$$\mathrm{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \qquad \mathrm{Recall}_c = \frac{TP_c}{TP_c + FN_c}, \qquad \mathrm{F1}_c = \frac{2\,\mathrm{Precision}_c\,\mathrm{Recall}_c}{\mathrm{Precision}_c + \mathrm{Recall}_c} \tag{5.9}$$

$$\mathrm{Acc}_{\mathrm{label}} = \frac{1}{5} \sum_{c=1}^{5} \frac{TP_c + TN_c}{N}, \qquad \mathrm{Acc}_{\mathrm{exact}} = \frac{1}{N} \sum_{n=1}^{N} \left[\mathbf{y}_n^{\mathrm{pred}} = \mathbf{y}_n \right] \tag{5.10}$$

Macro AUROC is the primary metric, because it does not depend on thresholds or class prevalence; its 95% confidence interval uses 1,000 bootstrap resamples of the test records [41], and differences between two models use a paired bootstrap on the same resamples.

5.1.9 Printed-ECG Reading (Photos, Scans and PDFs)

To accept pictures of paper ECGs, the project added ECG-Image-Database (Kaggle physionet/ecg-image-database, CC BY-ND 4.0), PTB-XL part only: real printouts of PTB-XL records that were scanned, photographed with phones, photographed off monitors, stained and creased. Folds and the test split are unchanged. synth.py draws synthetic printouts with a pixel-exact trace mask; segnet.py is a half-resolution U-Net trace segmenter trained only on those (4,000 steps on the CPU, validation Dice 0.853); digitize.py straightens the page, tracks the row baselines, follows the one-pixel centre line of each trace (Zhang–Suen thinning) and outputs the 12 × 1,000 signal with a quality score. finetune.py fine-tunes each client on the real photos of its own folds (13 per training fold, 104 in total), with early stopping, temperature, gate and triage on 60 fold-9 photos.

5.1.10 Web Portal and Voice Assistant

The CardioSentinel portal (web/) is a FastAPI back end with a static single-page front end. The patient app offers a home page with the latest heart check, 'Check my heart' (upload or example recordings), a plain-language result with the five findings and the rhythm strip, a symptom journal, 'Share with my doctor', appointments, a heart-health guide and a voice assistant. The doctor workspace offers an inbox of patient shares, a queue of ECGs to review, a 12-lead viewer with sign-off, patients, messages and appointments. The assistant (web/assistant.py) retrieves passages from the patient's own records with TF-IDF and asks a language model through OpenRouter to answer only from them, with an offline extractive fallback. Self-checks are private: a doctor sees a check, note or file only after the patient shares it.

5.2 Testing

5.2.1 Automated Tests

The repository contains 37 automated pytest tests in seven files (Table 5.4). They were run for this report with python -m pytest on Python 3.11.17: **37 passed** in 9.9 s.

Table 5.4 Automated test suite (python -m pytest, 37 passed)

Test file	Tests	What is checked	Result
test_splits.py	3	No patient in two splits; official folds 1–10; every record has a label	Pass
test_labels.py	1	SCP code → superclass mapping	Pass
test_signal.py	4	500 → 100 Hz shape; case-insensitive lead order; flat-lead normalisation and clipping; augmentation shapes	Pass
test_features.py	3	Prepared metadata carries only allow-listed features	Pass
test_fourclient.py	11	Client partitions disjoint and patient-separated; output shape of all four models; weights on the simplex for all three combiners; static weights find the informative client; metrics; triage bands meet targets and modes	Pass
test_images.py	9	Layout masks; masked normalisation; page mask; one-pixel thinning; digitizer recovers a known signal (3×4 and 12×1); blank page rejected; transparency; U-Net output size	Pass

Test file	Tests	What is checked	Result
test_web.py	5	Roles separated; patient sees only released results; clinician views; sign-up, self-check privacy and sharing; assistant answers from own records	Pass
Total	37		37 passed

5.2.2 Protocol and Leakage Checks

- **Patient separation.** test_splits.py and test_fourclient.py verify that no patient appears in more than one client's training data, the validation fold or the test fold.
- **Single use of the test fold.** evaluate.py and triage.py refuse a second full fold-10 run without --rerun, and all choices (combiner, thresholds, triage bands, ablations, explanations) were made on fold 9.
- **Validation versus test.** The selected gate scores 0.909 cross-validated on fold 9 and 0.905 on the test fold; the small gap shows that the validation-based selection did not over-fit.

5.2.3 Upload Smoke Test

Six fold-10 recordings (one single-label case per class, plus one MI + STTC case) were each submitted to the portal as a CSV, a rendered 3×4 PDF, a PNG and a photo-like JPG. The CSV results matched the Engine's direct predictions on the signal, every image path digitized without error, and, as designed, every photo finding was referred because photo triage is not yet validated. This test also found that PyMuPDF and OpenCV were missing from requirements.txt; both were added.

CHAPTER 6

PROJECT DEMONSTRATION

The system is started with the commands below; the portal then runs at http://127.0.0.1:8000. Demo accounts use the password demo1234: clinician@cardiosentinel.demo (doctor workspace) and patient@cardiosentinel.demo (patient app). With trained models present the portal runs in live mode: demo patients are seeded with real PTB-XL fold-10 recordings and real predictions.

```
pip install -r requirements.txt && pip install -e .
bash scripts/download.sh
python -m cardiomoe.data.ptbxl --prepare
python -m cardiomoe.fourclient.train --seeds 42 43 44
python -m cardiomoe.fourclient.aggregate --seeds 42 43 44
python -m cardiomoe.fourclient.evaluate --seeds 42 43 44
python -m cardiomoe.fourclient.triage --seeds 42 43 44
python web/server.py            # http://127.0.0.1:8000
```

The following screenshots were taken from the running portal.

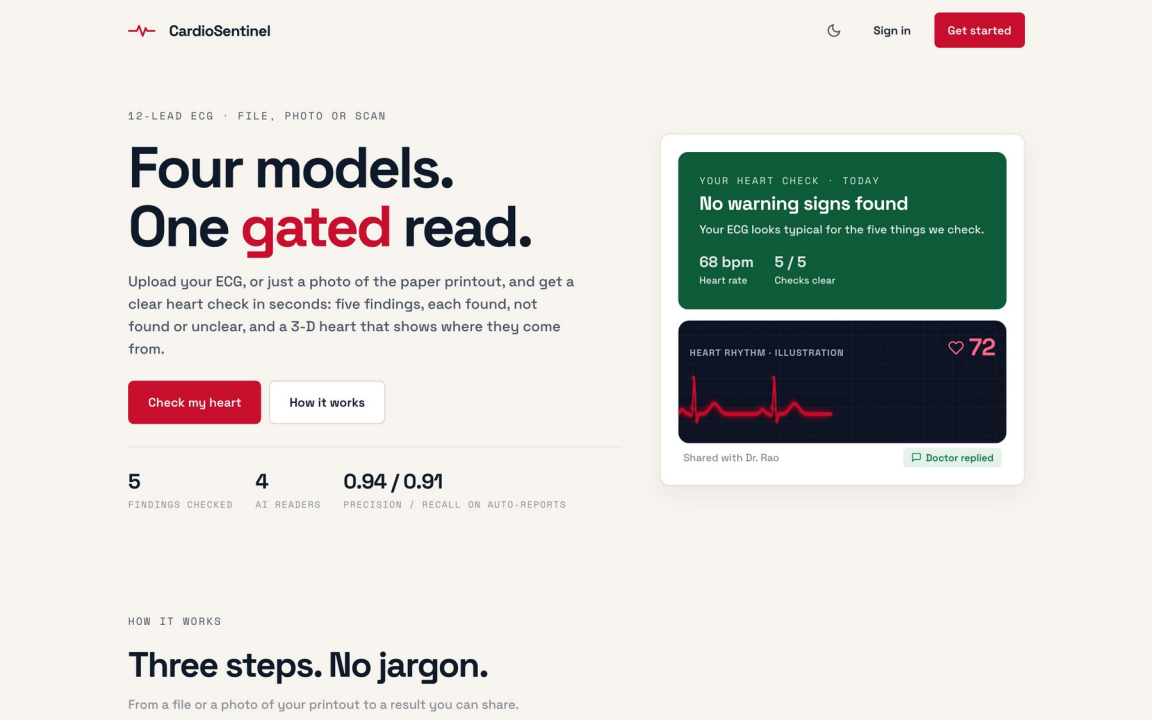


Figure 6.1 Welcome page of the CardioSentinel portal

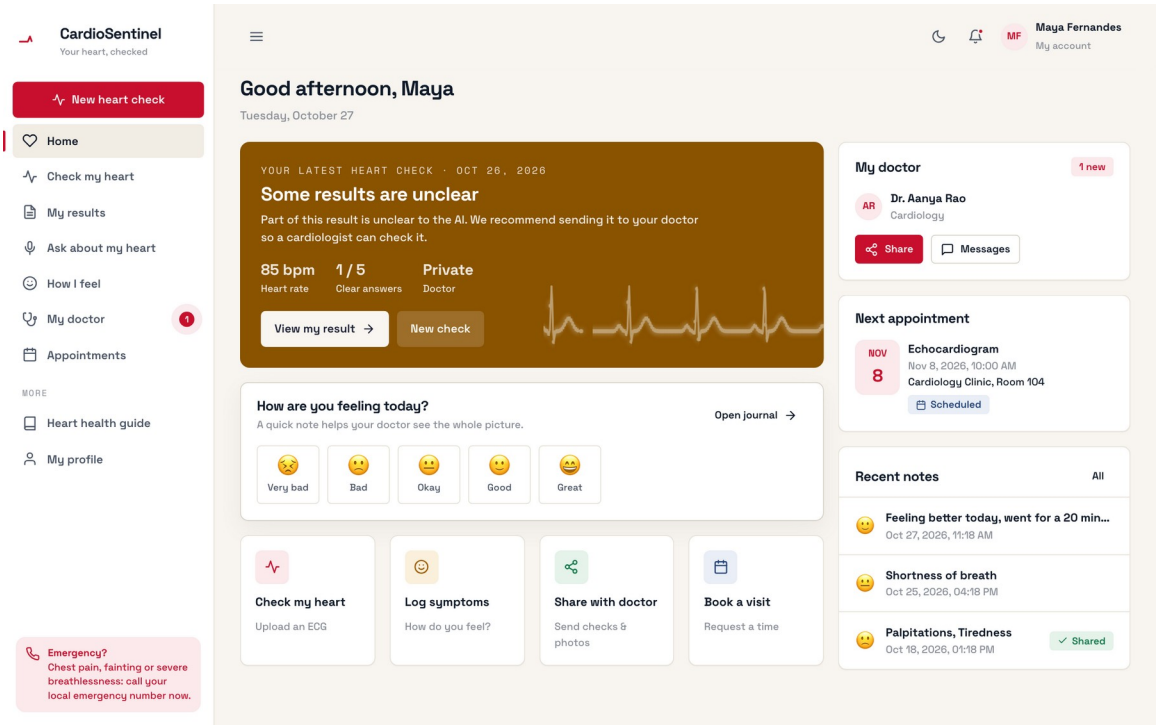


Figure 6.2 Patient home: latest heart check, doctor, next appointment and mood picker

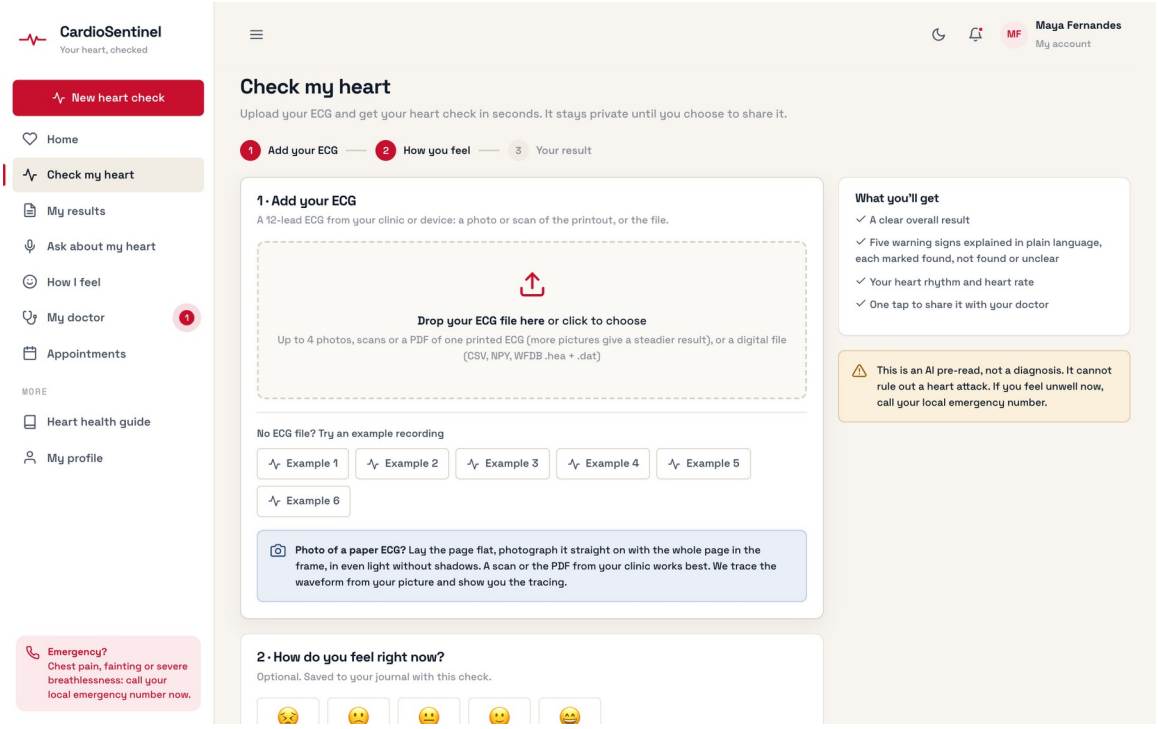


Figure 6.3 'Check my heart': upload an ECG file or up to four photos, scans or PDFs, or try an example recording

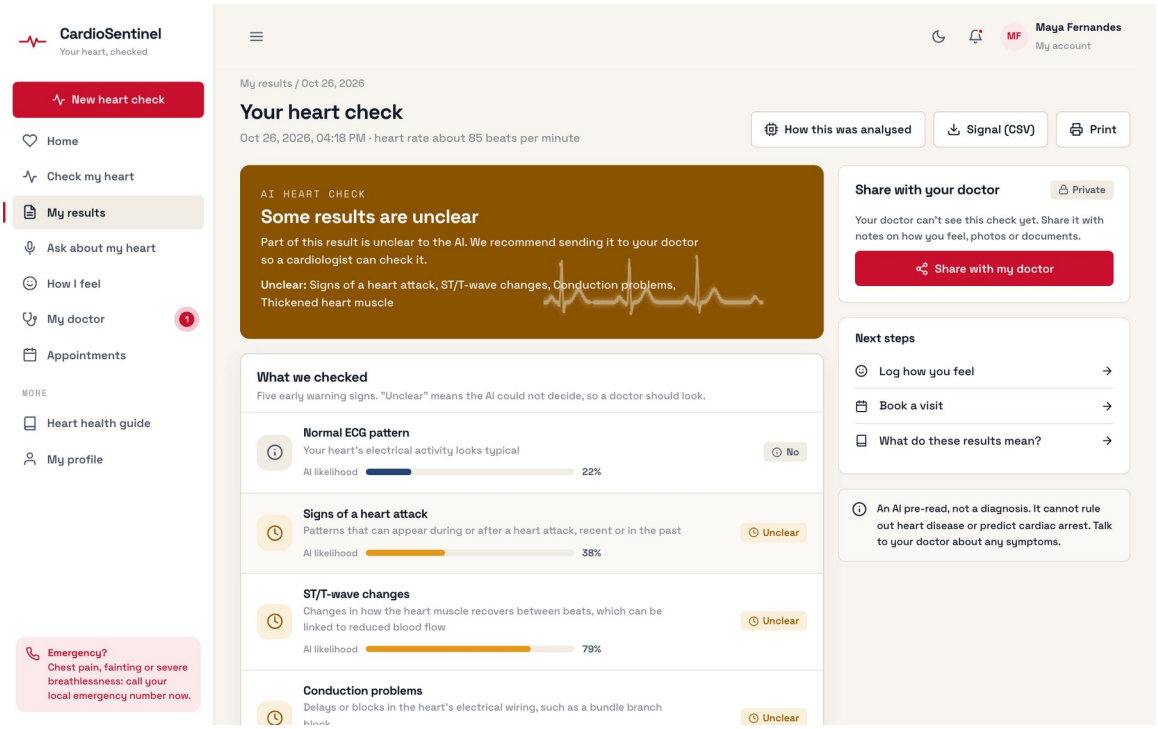


Figure 6.4 Result page: overall verdict, the five findings with their AI likelihood, and sharing with the doctor

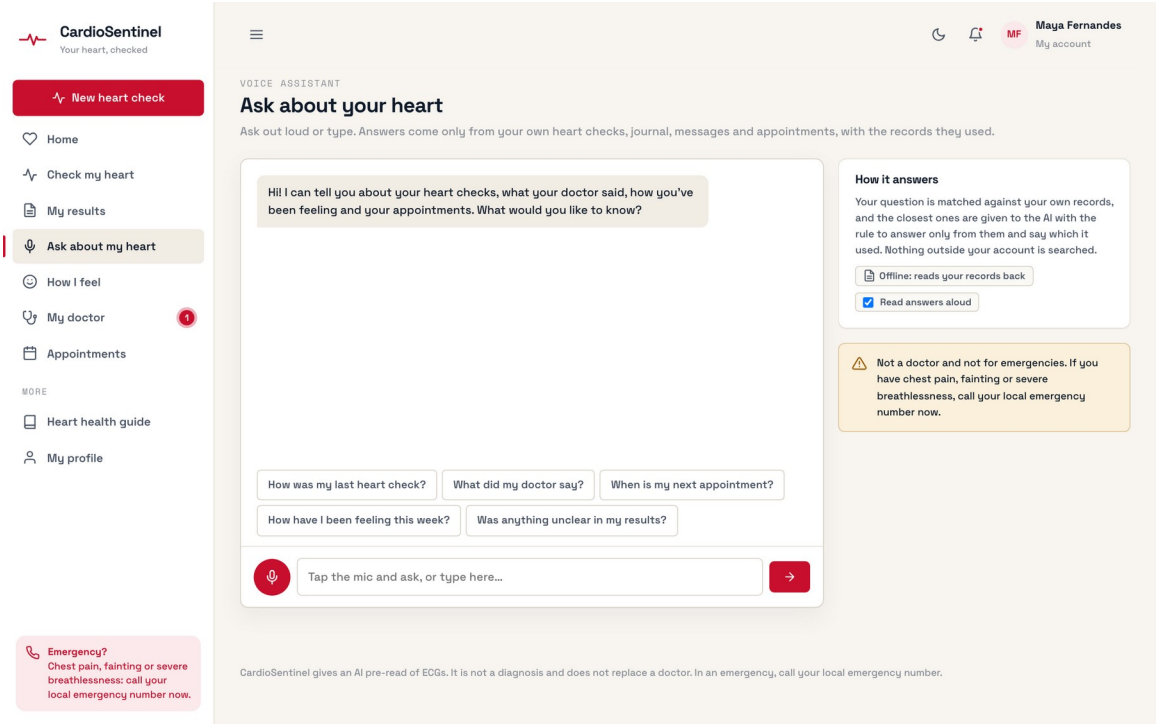


Figure 6.5 Voice assistant that answers from the patient's own records

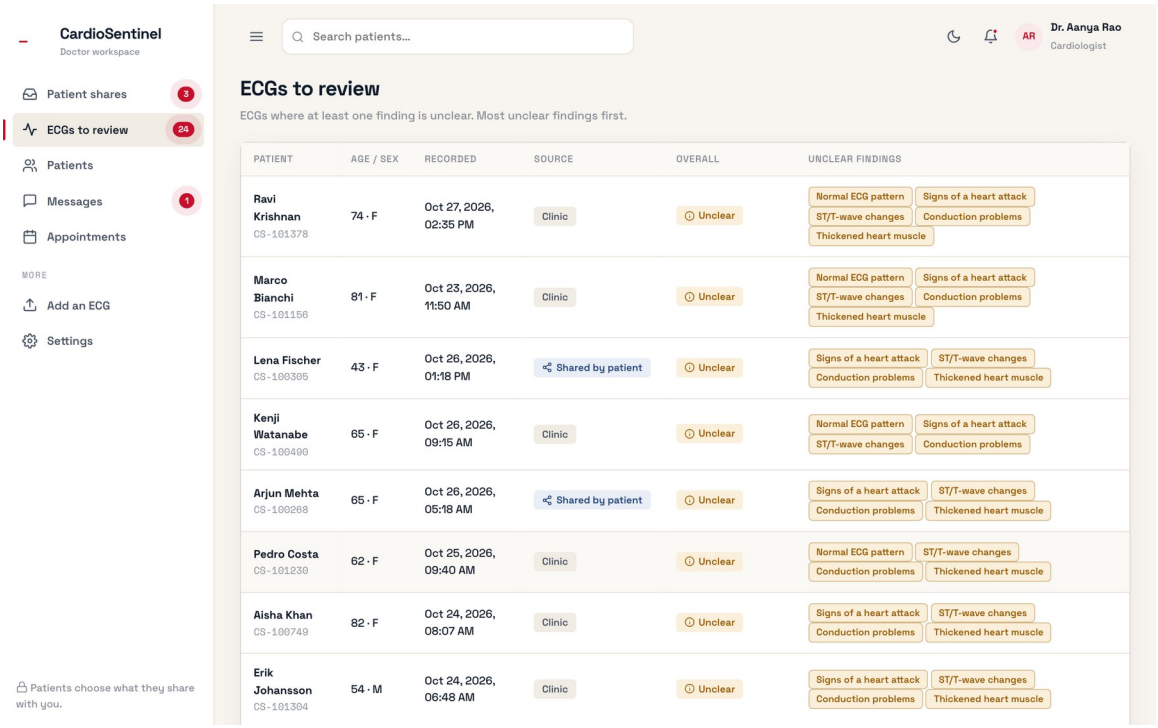


Figure 6.6 Doctor workspace: queue of ECGs to review with the unclear findings

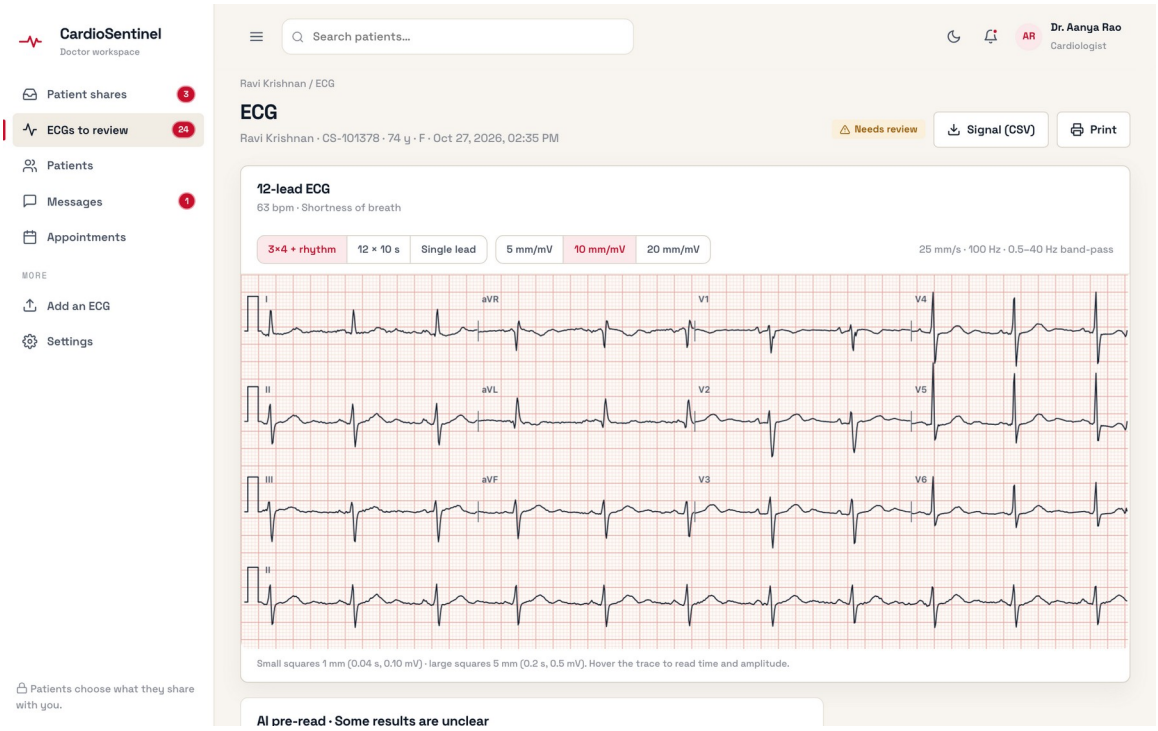


Figure 6.7 12-lead ECG viewer (3×4 + rhythm, 25 mm/s, 10 mm/mV) with sign-off

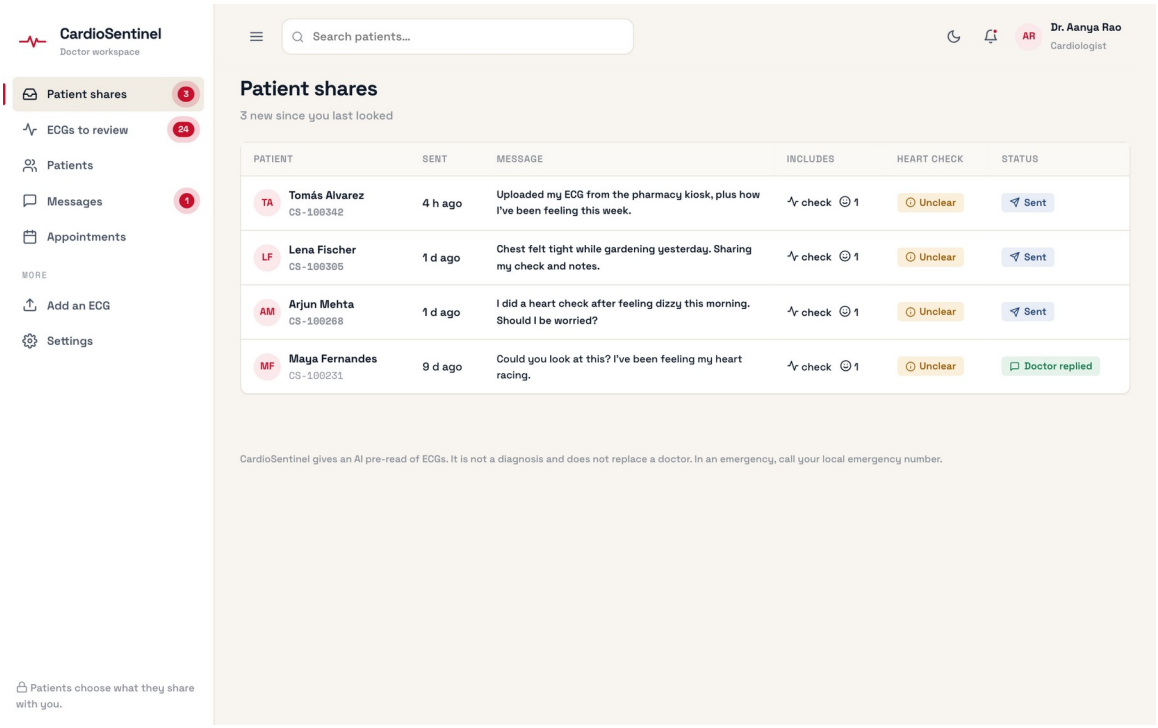


Figure 6.8 Patient shares inbox in the doctor workspace

CHAPTER 7

RESULT AND DISCUSSION

7.1 Dataset

All experiments use PTB-XL v1.0.3 [5], an open-access (CC BY 4.0) collection distributed through PhysioNet [37]: 21,799 clinical 12-lead ECGs of 10 s from 18,869 patients, annotated with SCP-ECG statements by up to two cardiologists. After removing 411 records without a diagnostic superclass, 21,388 records from 18,617 patients remain. The mean age is 59.4 ± 16.8 years, 48.1% of records belong to female patients, the mean number of labels per record is 1.30 and 24.1% of records are multi-label. Folds 1–8 (17,084 records, 80%) are the training data, fold 9 (2,146, 10%) the validation set and fold 10 (2,158 records from 1,877 patients, 10%) the test set (Table 7.1, Figure 7.1). NORM is the most frequent superclass (44.5%) and HYP the rarest (12.4%).

Table 7.1 Dataset details: records, patients and positive labels per partition (prevalence in % in parentheses)

Partition	Folds	Records	Patients	NORM	MI	STTC	CD	HYP
Client A	1, 2	4,277	3,676	1,907	1,087	1,050	966	527
Client B	3, 4	4,280	3,722	1,918	1,079	1,042	981	525
Client C	5, 6	4,264	3,725	1,869	1,125	1,060	990	535
Client D	7, 8	4,263	3,700	1,902	1,088	1,034	970	532
Train	1–8	17,084	14,823	7,596 (44.5)	4,379 (25.6)	4,186 (24.5)	3,907 (22.9)	2,119 (12.4)
Validation	9	2,146	1,917	955 (44.5)	540 (25.2)	528 (24.6)	495 (23.1)	268 (12.5)
Test	10	2,158	1,877	963 (44.6)	550 (25.5)	521 (24.1)	496 (23.0)	262 (12.1)
Total	1–10	21,388	18,617	9,514 (44.5)	5,469 (25.6)	5,235 (24.5)	4,898 (22.9)	2,649 (12.4)

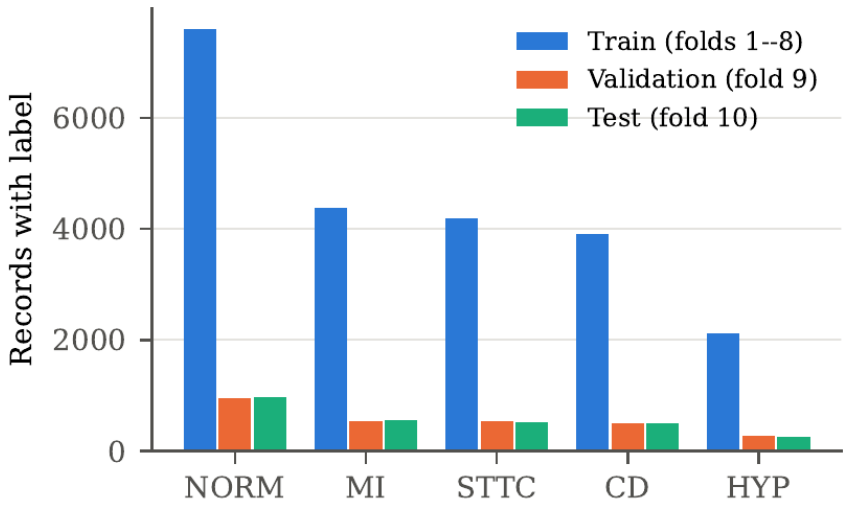


Figure 7.1 Number of records carrying each diagnostic superclass in the training, validation and test folds

7.2 Comparison with Traditional Machine Learning and Single Clients

Table 7.2 compares the proposed model with traditional classifiers on 172 hand-crafted

features, the individual deep clients, the learned aggregation variants and a centrally trained reference. All rows were evaluated once on the same test fold with thresholds chosen on fold 9.

Table 7.2 Comparison with traditional machine-learning models, the individual deep clients and the aggregation variants on the PTB-XL test fold (mean ± SD over 3 seeds; Acc. = label-wise accuracy; Prec., Rec., F1, AUROC and AUPRC are macro averages over the five superclasses)

Model	**Params**	**Acc. (%)**	**Prec. (%)**	**Rec. (%)**	**F1 (%)**	**AUROC**	**AUPRC**
Traditional machine learning (172 hand-crafted features, pooled folds 1–8)							
Logistic Regression	–	84.4 ± 0.0	65.3 ± 0.0	71.7 ± 0.0	68.2 ± 0.0	0.880 ± 0.000	0.724 ± 0.000
SVM (RBF)	–	85.8 ± 0.0	67.3 ± 0.0	74.5 ± 0.0	70.6 ± 0.0	0.898 ± 0.000	0.772 ± 0.000
Random Forest	–	86.2 ± 0.2	69.1 ± 0.8	74.0 ± 1.2	71.2 ± 0.2	0.902 ± 0.001	0.778 ± 0.001
KNN	–	82.2 ± 0.0	60.8 ± 0.0	68.9 ± 0.0	64.4 ± 0.0	0.859 ± 0.000	0.700 ± 0.000
XGBoost	–	86.8 ± 0.0	69.7 ± 0.2	76.3 ± 0.5	72.6 ± 0.2	0.909 ± 0.002	0.789 ± 0.003
Single deep clients (25% of training data each)							
Client A: 1D CNN	0.44M	84.8 ± 0.2	64.6 ± 0.6	72.6 ± 0.8	68.0 ± 0.3	0.888 ± 0.002	0.738 ± 0.005
Client B: 1D ResNet	6.73M	84.0 ± 0.8	63.0 ± 1.3	73.4 ± 1.8	67.2 ± 0.4	0.882 ± 0.002	0.721 ± 0.005
Client C: BiLSTM	0.67M	85.5 ± 0.3	66.2 ± 1.2	73.3 ± 0.5	69.2 ± 0.3	0.894 ± 0.000	0.745 ± 0.003
Client D: CNN-LSTM	0.47M	85.7 ± 0.8	67.0 ± 2.8	72.2 ± 3.1	69.0 ± 0.1	0.896 ± 0.001	0.749 ± 0.006
Aggregation of the four clients							
Static learned weights	4	86.6 ± 0.3	68.5 ± 0.9	72.8 ± 0.8	70.3 ± 0.3	0.903 ± 0.001	0.764 ± 0.004
Dynamic gate	≈1k	86.4 ± 0.3	68.1 ± 1.2	72.9 ± 1.3	70.2 ± 0.1	0.901 ± 0.001	0.759 ± 0.002
Proposed (equal-weight gate)	8.31M	86.9 ± 0.2	69.8 ± 0.9	72.5 ± 2.1	70.7 ± 0.6	0.905 ± 0.000	0.767 ± 0.002
Centralised reference (data pooled, not federated)							
CNN-LSTM on folds 1–8	0.47M	86.4 ± 0.7	68.5 ± 1.2	74.0 ± 0.6	70.7 ± 0.8	0.909 ± 0.001	0.774 ± 0.002

The four deep clients, each trained on a quarter of the data, reach test macro AUROCs between 0.882 (Client B, 1D ResNet) and 0.896 (Client D, CNN-LSTM). The ResNet, despite the most parameters (6.73M), is the weakest and stops early after 13.7 epochs on average, a sign of over-fitting on 4,280 records. The proposed equal-weight gate raises the macro AUROC to 0.905 ± 0.000, the label-wise accuracy to 86.9% (from at most 85.7%), the macro F1 to 70.7% (from at most 69.2%) and the macro precision to 69.8%, while keeping the recall (72.5%) at the level of the clients. A CNN-LSTM trained centrally on all eight folds attains 0.909 ± 0.001 and the same macro F1, so the federated gate recovers about 70% of the AUROC lost by splitting the data and ties the pooled model in F1.

The traditional classifiers are strong baselines. On pooled data XGBoost reaches 0.909 ± 0.002 and a macro F1 of 72.6%, because its amplitude features keep the absolute voltages that per-recording z-scoring removes. Trained on a single client's folds, however, XGBoost reaches 0.898 and every locally trainable model stays below the federated gate (Table 7.3). Per class, the gate

has the highest AUROC in every superclass, with its largest gain on the hardest class, HYP (0.828 versus at most 0.809; Table 7.4, Figure 7.2).

Table 7.3 Traditional classifiers trained locally (one client's folds; mean ± SD over clients A–D, seed 42) or on pooled data (mean ± SD over 3 seeds), compared with the best deep clients (test fold)

Model	Local (one client, mean of A–D) AUROC	Local (one client, mean of A–D) F1 (%)	Pooled (folds 1–8) AUROC	Pooled (folds 1–8) F1 (%)
Logistic Regression	0.870 ± 0.001	67.0 ± 0.1	0.880 ± 0.000	68.2 ± 0.0
SVM (RBF)	0.886 ± 0.002	68.8 ± 0.3	0.898 ± 0.000	70.6 ± 0.0
Random Forest	0.894 ± 0.001	69.8 ± 0.2	0.902 ± 0.001	71.2 ± 0.2
KNN	0.847 ± 0.004	63.3 ± 0.6	0.859 ± 0.000	64.4 ± 0.0
XGBoost	0.898 ± 0.002	70.4 ± 0.4	0.909 ± 0.002	72.6 ± 0.2
Deep client C (BiLSTM)	0.894 ± 0.000	69.2 ± 0.3	–	–
Deep client D (CNN-LSTM)	0.896 ± 0.001	69.0 ± 0.1	–	–

Table 7.4 Single clients versus the federated gate: test AUROC per superclass (mean of 3 seeds; best in bold)

Model	NORM	MI	STTC	CD	HYP	Macro
Client A (1D CNN)	0.926	0.891	0.925	0.900	0.798	0.888
Client B (1D ResNet)	0.922	0.877	0.918	0.884	0.808	0.882
Client C (BiLSTM)	0.931	0.901	0.926	0.902	0.809	0.894
Client D (CNN-LSTM)	0.933	0.906	0.923	0.909	0.808	0.896
Proposed	**0.938**	**0.912**	**0.933**	**0.914**	**0.828**	0.905

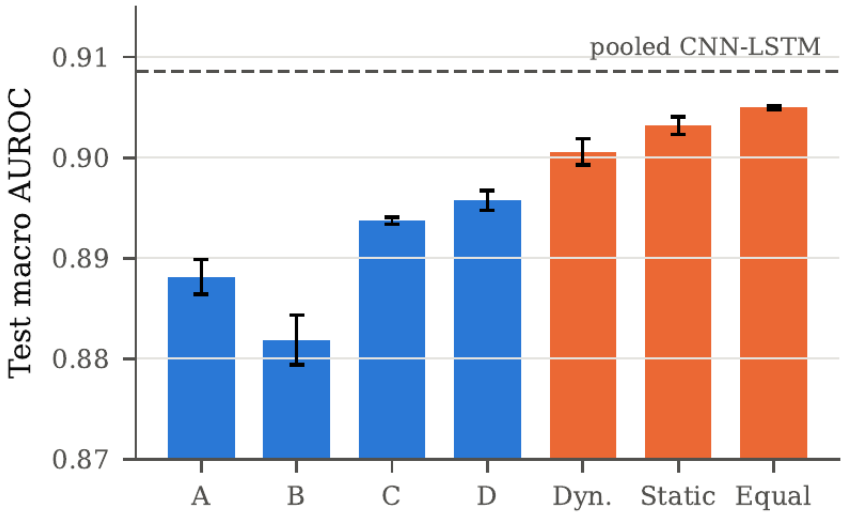


Figure 7.2 Test macro AUROC of the four clients and the three aggregation variants (mean ± SD over 3 seeds)

The gain over the best single client is statistically significant. In a paired bootstrap on the test fold the gate exceeds the best client chosen on fold 9 by +0.012 [+0.009, +0.016] AUROC for seed 42, +0.011 [+0.007, +0.014] for seed 43 and +0.009 [+0.005, +0.013] for seed 44; the macro F1 improves by +0.018, +0.010 and +0.024, with the interval excluding zero in two of three seeds.

7.3 Ablation Experiments

Client subsets. Every one of the 15 non-empty subsets of {A, B, C, D} was combined with equal weights (Table 7.5). The full federation is best on both splits (fold 9: 0.909; test: 0.905), every pair beats every single client, and returns diminish (single at most 0.896, pairs 0.903, triples 0.905). The Shapley values (Table 7.6) rank C and D highest but are of the same order (0.097–0.105 on the test fold). This explains why equal weighting works: the learned static weights give Client A almost nothing (0.001), although its marginal contribution is comparable.

Table 7.5 Client-subset ablation (equal-weight gate; mean over 3 seeds, test SD shown)

#Clients	Subset	Fold-9 CV AUROC	Test AUROC	Δ vs. ABCD
4	**ABCD**	0.909	0.905 ± 0.000	0.000
3	ACD	0.908	0.905 ± 0.000	0.000
3	BCD	0.908	0.904 ± 0.001	-0.001
3	ABC	0.906	0.901 ± 0.000	-0.004
3	ABD	0.905	0.902 ± 0.001	-0.003
2	CD	0.907	0.903 ± 0.001	-0.002
2	AC	0.905	0.900 ± 0.000	-0.005
2	AD	0.904	0.901 ± 0.001	-0.004
2	BC	0.903	0.898 ± 0.001	-0.007
2	BD	0.903	0.900 ± 0.001	-0.005
2	AB	0.899	0.895 ± 0.001	-0.010
1	C	0.900	0.894 ± 0.000	-0.011
1	D	0.898	0.896 ± 0.001	-0.009
1	A	0.893	0.888 ± 0.002	-0.017
1	B	0.886	0.882 ± 0.003	-0.023

Table 7.6 Shapley value φi of each client for the macro AUROC of the equal-weight federation (v(∅)=0.5) and the mean learned static weight

Client	Architecture	φ (fold 9)	φ (test)	Static weight wi
A	1D CNN	0.1009	0.0998	0.001
B	1D ResNet	0.0980	0.0971	0.042
C	BiLSTM	0.1055	0.1033	0.549
D	CNN-LSTM	0.1043	0.1047	0.407
Sum	–	0.4087	0.4050	1.000

Fusion strategy. Equal weights reach the highest cross-validated macro AUROC (0.909) and were selected in all three seeds; static weights (0.907) and the dynamic gate (0.904) did not reach the margin (Table 7.7). The learned combiners have a lower BCE (better calibration), but they concentrate the weight on Clients C and D (Figure 7.3) and lose diversity. The same ordering holds on the test fold (0.905, 0.903, 0.901), confirming the selection. A later re-examination on fold 9 (logit mean 0.9089, per-class weights 0.9047, stacked logistic regression 0.9075 and 0.9084 per class, versus equal 0.9087) found nothing that clears the 0.002 rule.

Table 7.7 Ablation of the fusion strategy (fold-9 CV values used for selection; test values mean ± SD over 3 seeds)

Fusion strategy	CV AUROC	CV BCE	Test AUROC	Test F1 (%)	Exact match (%)
Equal weights (selected)	0.909	0.372	0.905 ± 0.000	70.7 ± 0.6	58.1 ± 0.7
Static learned weights	0.907	0.346	0.903 ± 0.001	70.3 ± 0.3	57.0 ± 1.5
Dynamic per-patient gate	0.904	0.338	0.901 ± 0.001	70.2 ± 0.1	56.8 ± 0.7

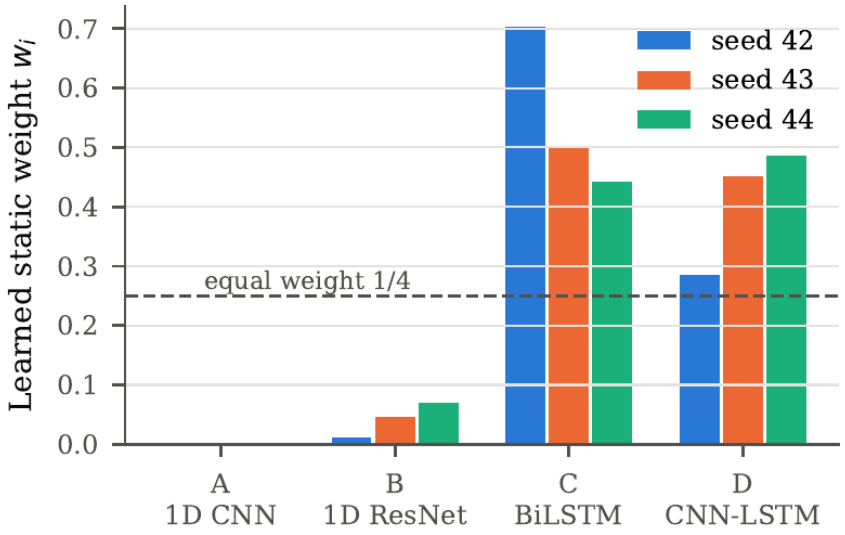


Figure 7.3 Static aggregation weights learned on fold 9 for each seed; the dashed line marks the equal weight 1/4

Activation function and optimizer. On Client D (seed 42, fold 9 only) ReLU and LeakyReLU perform identically (0.898) and the activation changes the AUROC by at most 0.004 (Table 7.8). The optimizer matters much more: AdamW and Adam reach 0.898, whereas SGD with momentum reaches 0.786 and RMSprop 0.855 at the same maximum learning rate (Table 7.9).

Table 7.8 Activation-function ablation (Client D, CNN-LSTM, seed 42; fold-9 macro AUROC)

Activation	Fold-9 AUROC	Best / run epochs	Train loss
ReLU (used)	0.898	23 / 31	0.444
LeakyReLU	**0.898**	24 / 32	0.423
ELU	0.895	14 / 22	0.547
GELU	0.896	14 / 22	0.534
Tanh	0.894	19 / 27	0.484

Table 7.9 Optimizer ablation (Client D, CNN-LSTM, seed 42; fold-9 macro AUROC; maximum learning rate 10^{-3} for all optimizers)

Optimizer	Fold-9 AUROC	Best / run epochs	Train loss
AdamW (used)	**0.898**	23 / 31	0.444
Adam [42]	0.898	20 / 28	0.480
SGD (momentum 0.9)	0.786	30 / 38	1.014
RMSprop (momentum 0.9)	0.855	3 / 11	0.724

7.4 Federated Learning Analysis

Because the federation has a single round, its training dynamics are those of the four local trainings (Figure 7.4, Table 7.10). The convolutional clients converge fastest (best epochs 12.7 and 13.7) and then over-fit; the recurrent clients learn more slowly but keep improving (27.0 and 22.7) and end with the highest validation AUROCs (0.900 and 0.898). All fitted temperatures exceed one (1.25–1.74), so every client is over-confident and calibration is needed before averaging. Figure 7.5 shows that the global model improves monotonically with the number of

aggregated clients and flattens after three.

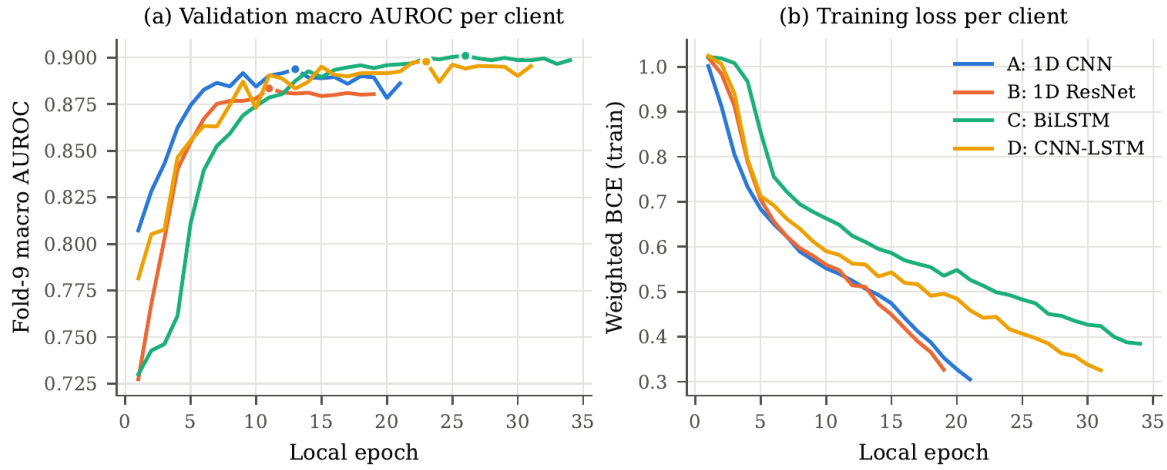


Figure 7.4 Local training of the four clients (seed 42): fold-9 macro AUROC and class-weighted training BCE per epoch

Table 7.10 Client-wise training summary (mean over 3 seeds; loss and AUROC mean ± SD at the selected epoch)

Model	ntrain	Epochs run	Best epoch	Train loss	Fold-9 AUROC	Ti
Client A	4,277	20.7	12.7	0.510 ± 0.036	0.893 ± 0.001	1.74 ± 0.22
Client B	4,280	21.7	13.7	0.479 ± 0.079	0.886 ± 0.002	1.52 ± 0.15
Client C	4,264	35.0	27.0	0.466 ± 0.017	0.900 ± 0.001	1.25 ± 0.09
Client D	4,263	30.7	22.7	0.450 ± 0.024	0.898 ± 0.001	1.35 ± 0.06
Pooled (folds 1–8)	17,084	31.0	23.0	0.452 ± 0.037	0.914 ± 0.002	1.31 ± 0.08

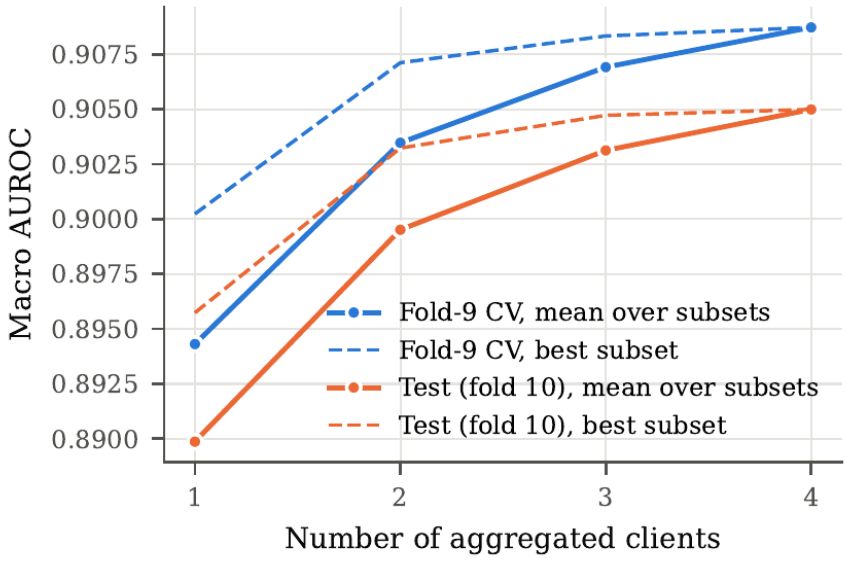


Figure 7.5 Macro AUROC of the global (server) model as a function of the number of aggregated clients

Comparison with other FL frameworks. Parameter averaging cannot combine heterogeneous clients, so FedAvg, FedProx (μ = 0.01), SCAFFOLD and FedNova were run with one shared architecture (the CNN-LSTM) on the same four partitions for 20 rounds of one local epoch (Table 7.11, Figure 7.6). FedAvg and FedNova reach 0.907 and FedProx 0.906, slightly above the one-shot gate (0.905) and close to centralised training (0.909). SCAFFOLD failed with

AdamW local steps: its best model was the one after round 1 (0.842). The proposed model therefore trades about 0.002 AUROC for a single communication round, five probabilities per client and query, and a free choice of architecture.

Table 7.11 Comparison with federated-learning frameworks on the same four client partitions (seed 42; test fold unless stated). [a]Patient-grouped CV on fold 9.

Method	Rounds	Same arch.	Fold-9 AUROC	Test AUROC	F1 (%)	Acc. (%)
Local only (best client, D)	0	–	0.898	0.895	68.9	85.0
FedAvg [14]	20/20	yes	0.911	0.907	71.8	87.2
FedProx (μ=0.01) [15]	20/20	yes	0.910	0.906	71.4	86.9
SCAFFOLD [16]	1/20	yes	0.850	0.842	62.4	80.6
FedNova [43]	20/20	yes	0.911	0.907	71.5	86.8
Proposed (one-shot gate)	1	no	0.908[a]	0.905	70.9	86.8
Centralised (pooled data)	–	–	0.916	0.909	69.6	85.5

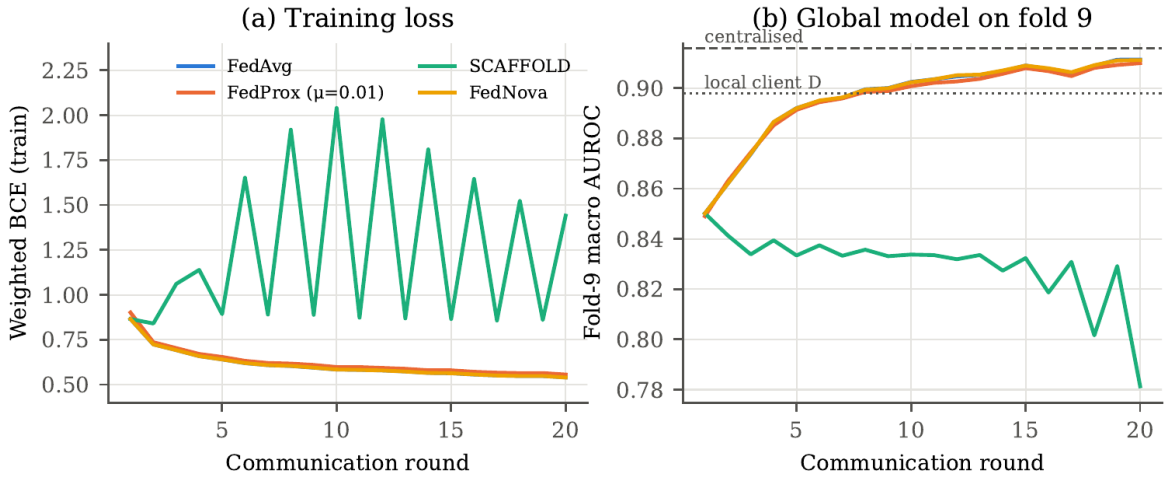


Figure 7.6 Convergence of the parameter-averaging frameworks (shared CNN-LSTM, seed 42)

7.5 Class-wise Analysis and Triage

Figure 7.7 shows the per-class confusion matrices for seed 42 at the fold-9 thresholds (0.42 for NORM, 0.62 for MI and STTC, 0.64 for CD and HYP) and Figure 7.8 the ROC curves. NORM is detected most reliably (887 of 963 positives), CD is predicted conservatively (58 false positives, 177 of 496 positives missed) and HYP is the most difficult class (129 true positives, 133 false negatives), because hypertrophy criteria rely on voltage amplitudes that per-recording z-scoring partly removes. Table 7.12 gives the class-wise metrics over three seeds: every class except HYP has an AUROC above 0.91.

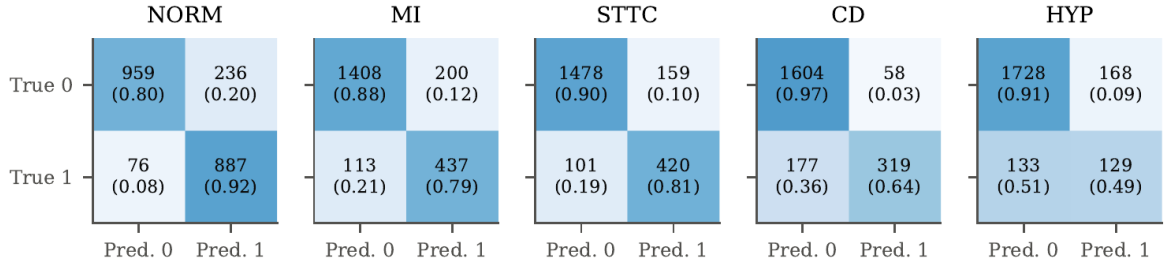


Figure 7.7 Per-class confusion matrices of the proposed model on the test fold (seed 42)

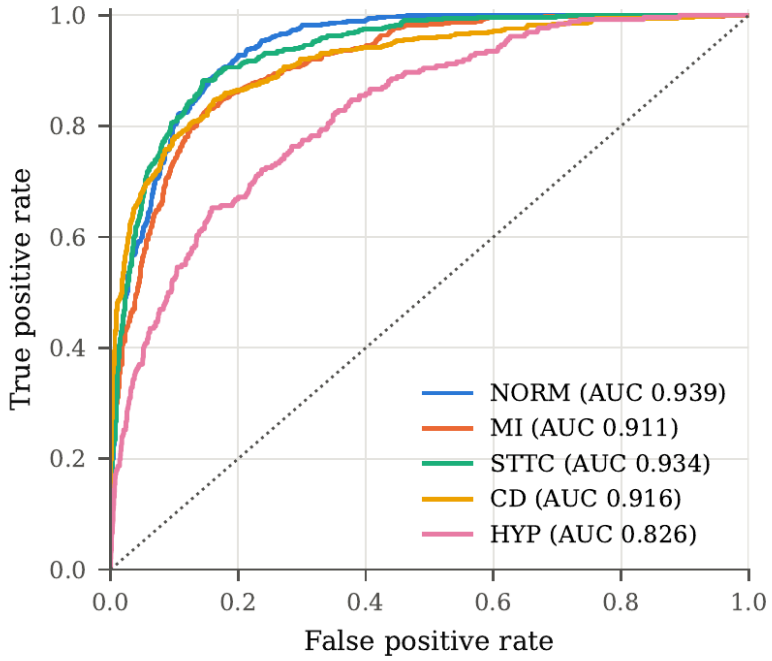


Figure 7.8 ROC curves of the proposed model on the test fold (seed 42)

Table 7.12 Class-wise performance of the proposed model on the test fold (mean of 3 seeds)

Class	Precision	Recall	F1-score	AUROC	Support
NORM	0.782	0.930	0.850	0.938	963
MI	0.724	0.730	0.725	0.912	550
STTC	0.745	0.792	0.768	0.933	521
CD	0.800	0.682	0.735	0.914	496
HYP	0.439	0.490	0.461	0.828	262
Macro avg.	0.698	0.725	0.707	0.905	2792
Weighted avg.	0.735	0.780	0.753	0.917	2792

Triage. Applying the rule-in / rule-out bands fitted on fold 9 to the test fold, 53.7% of all findings are decided automatically with a macro precision of 0.937, a macro recall of 0.908 over the automated classes and an accuracy of 0.967; 17.8% of the ECGs are decided completely without referral. NORM, MI and CD were automated in all seeds, STTC in one of three seeds and HYP only in rule-out mode. Not every target was met: the test recall of CD was 0.880. Stricter targets trade coverage for precision and recall on fold 9: a 0.95 target automates 39% of findings at 0.976 / 0.973 and a 0.97 target 33% at 0.995 / 0.994.

7.6 Explainability Results

Explanations were computed for the seed-42 equal-weight gate on 300 random fold-9 records; the test fold was not used. The attributions are spread over all twelve leads (none above 11.0%; uniform 8.3%), but the emphasis follows the expected anatomy (Figure 7.9): for MI aVF, V2, II and V3; for HYP V6, V5, aVR and V1, the leads of the classical voltage criteria; for CD II, aVR, V6 and V1, where bundle branch blocks are recognised. Local explanations (Figure 7.10) peak at the QRS complexes and ST segments rather than the baseline. The deletion test (Table 7.13) confirms faithfulness for NORM, MI and CD (removing the top-3 leads lowers the probability by 0.135, 0.057 and 0.070 against 0.074, 0.006 and 0.009 for random leads); for STTC

and HYP the evidence is redundant across leads.

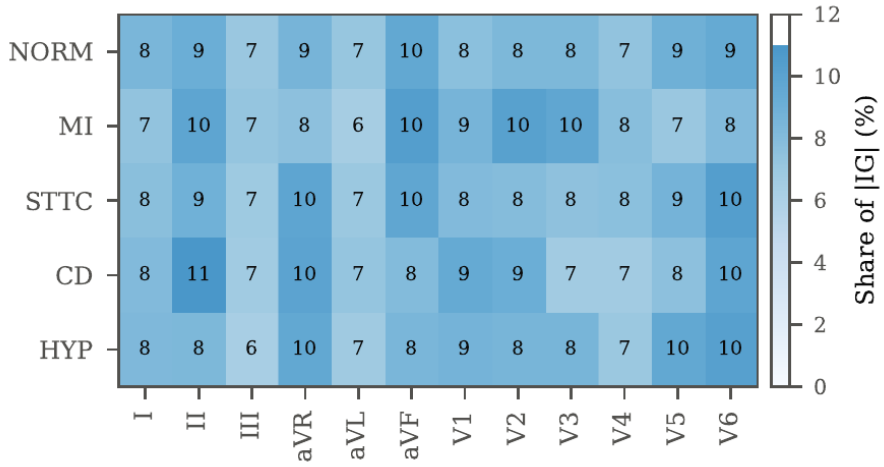


Figure 7.9 Global explanation: share of the absolute Integrated-Gradients attribution per lead (%) for each superclass

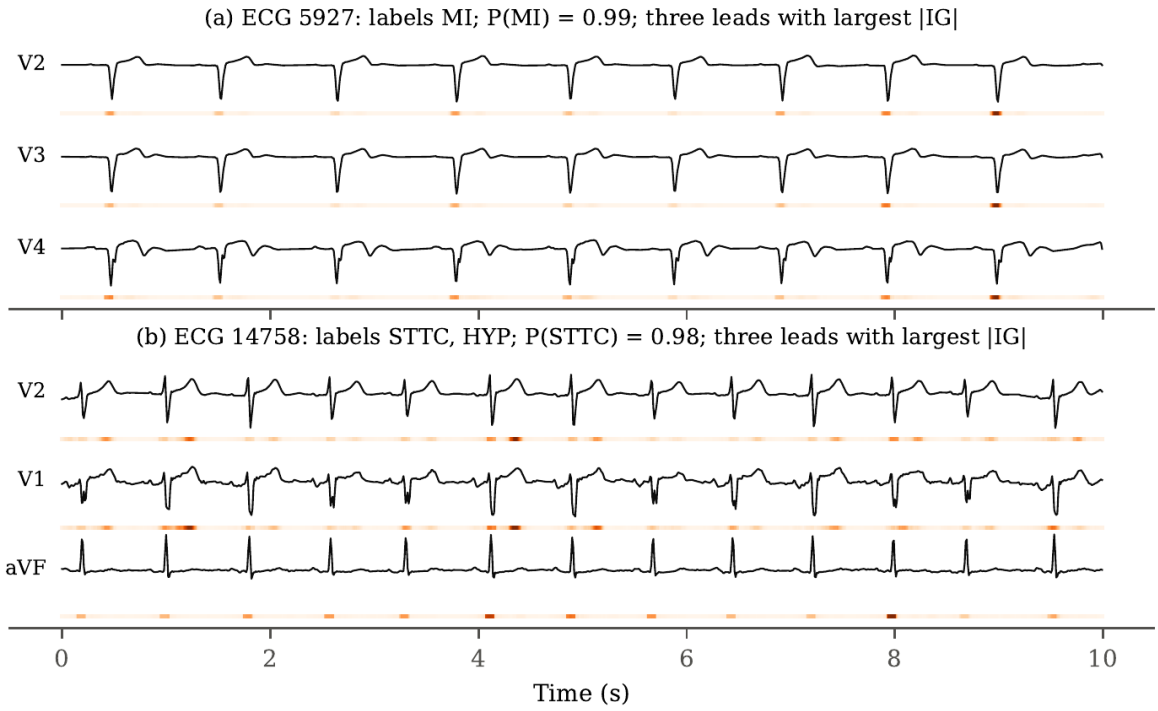


Figure 7.10 Local explanations for the most confident true-positive MI and STTC validation records

Table 7.13 Deletion test: mean gate probability of the true class after zeroing the k leads with the largest IG attribution or k random leads (fold-9 positives, seed 42)

Class	n	Mean p (intact)	Top-k IG leads removed k=1	Top-k IG leads removed k=3	Random k leads removed k=1	Random k leads removed k=3
NORM	131	0.741	0.713	0.606	0.722	0.667
MI	81	0.750	0.720	0.693	0.744	0.744
STTC	65	0.764	0.765	0.772	0.764	0.771
CD	80	0.758	0.715	0.688	0.746	0.749
HYP	31	0.620	0.626	0.621	0.627	0.613

7.7 Reading Photos of Paper ECGs

The digitizer's median per-lead correlation with the recorded signal on fold 10 is 0.78 for digital red grids, 0.44 for colour scans, 0.34 for phone photos and monitor photos, 0.19 for grayscale scans, 0.18 for green grids, 0.12 for stained scans and 0.00 for stained phone photos; 98% of the photos were digitized. Table 7.14 gives the one-time fold-10 evaluation on 100 real photos of 62 records.

Table 7.14 Real fold-10 photos (100 photos, 62 records, seed 42, evaluated once)

Model	Macro AUROC (95% CI by record)	Macro F1	Exact match
Image gate (static weights, almost all on D)	0.841 (0.746–0.916)	0.558	0.388
Signal gate on the same digitized photos (no fine-tuning)	0.859 (0.792–0.919)	0.343	0.082
Signal gate on the recorded signal of the same records	0.946 (0.906–0.975)	0.696	0.684

Fine-tuning on 104 photos did not raise the AUROC over the untuned models (0.841 versus 0.859), but it fixed the thresholds (F1 and exact match). Photo triage fitted on 59 fold-9 photos decides 50% of findings at an accuracy of 0.943, but with a macro precision of 0.873 and recall of 0.812 (MI precision 0.60), below the 0.90 target. The portal therefore refers every photo finding to a doctor until the bands are fitted on more validation photos.

7.8 Comparison with the State of the Art

Table 7.15 compares the proposed model with published results on the PTB-XL superclass task. The models of [6] were trained centrally on the same folds and are the most comparable. The proposed model does not exceed the best centrally trained models: its 0.905 is 0.016–0.029 below them. Part of the gap comes from the data restriction (each client sees 25% of the records) and part from the training recipe, since our own pooled CNN-LSTM reaches 0.909; the recipe was kept fixed for all models to isolate the effect of federation.

Table 7.15 Comparison with state-of-the-art results on the PTB-XL five-superclass task

Method	Training data	Macro AUROC	Acc. (%)
Wavelet + NN [6]	pooled	0.874	–
inception1d [6]	pooled	0.921	–
LSTM (bidir.) [6]	pooled	0.921	–
xresnet1d101 [6]	pooled	0.928	–
resnet1d_wang [6]	pooled	0.930	–
Ensemble [6]	pooled	0.934	–
ST-MEM [8]	pooled + pretrain	0.933	82.5
METS, zero-shot [10]	pooled + text	–	84.2
Pooled CNN-LSTM (ours)	pooled	0.909	86.4 (label)
XGBoost, 172 features (ours)	pooled	0.909	86.8 (label)
FedAvg, CNN-LSTM (ours, seed 42)	4 sites, 20 rounds	0.907	87.2 (label)
Proposed (federated)	4 sites, 1 round	**0.905**	**86.9 (label)**

Strengths. The approach delivers most of the benefit of pooling without moving any record or parameter, outperforms every model that can be trained at a single site, comes within 0.002 AUROC of FedAvg with one communication step and free architectures, is significantly

better than the best institution in every seed and is highly reproducible (SD of the macro AUROC below 0.001). **Limitations.** The four clients are simulated partitions of one dataset with near-identical label distributions; fold 9 is used for early stopping, calibration, thresholds and the gate, so validation figures are optimistic; the learned gates did not beat equal weighting; HYP remains weak; FedAvg-type baselines and pooled XGBoost are stronger; the FL baselines and ablations used a single seed; and photo reading is not yet accurate enough for automatic reports.

7.9 Cost Analysis

The project has no licence or data cost and used no paid cloud resources (Table 7.16). The compute time of the main experiment follows from the measured epoch times and the mean number of epochs run (Table 7.10): Client A 20.7 × 26.8 s ≈ 9.2 min, Client B 21.7 × 38.5 s ≈ 13.9 min, Client C 35.0 × 65.4 s ≈ 38.2 min and Client D 30.7 × 28.2 s ≈ 14.4 min, i.e. about 76 minutes per seed and about 3.8 hours for the three seeds on an M1 MacBook Air. Because the clients train independently, real institutions would train in parallel, so the wall-clock time of the federation is that of the slowest client (about 38 minutes). Communication is one round; at query time each client returns five numbers.

Table 7.16 Cost analysis

Item	Cost
PTB-XL v1.0.3 (CC BY 4.0), ECG-Image-Database (CC BY-ND 4.0)	Free (open access)
Python, PyTorch, scikit-learn, XGBoost, FastAPI, SQLite, Tectonic	Free (open source)
Hardware	Existing Apple M1 MacBook Air (8 GB); no GPU server
Training compute, 4 clients × 3 seeds	≈ 3.8 h on the laptop (≈ 76 min per seed)
Voice assistant language model (optional)	Pay-per-use via OpenRouter; offline fallback is free
Communication per ECG at inference	20 numbers (5 probabilities × 4 clients)

CHAPTER 8

CONCLUSION

This project built CardioSentinel, a privacy-preserving, one-shot federated model for the multi-label classification of 12-lead ECGs into the five PTB-XL diagnostic superclasses, and deployed it in a patient and clinician portal. Four clients trained a 1D CNN, a 1D ResNet, a BiLSTM and a CNN-LSTM on disjoint, patient-separated quarters of the training data, calibrated them, and shared only their class probabilities with a server that combines them on the probability simplex. A parsimony-first selection on patient-grouped validation predictions chose equal weighting in all three seeds.

On the untouched test fold the federated model reached a macro AUROC of 0.905 ± 0.000, a macro F1 of 70.7% and a label-wise accuracy of 86.9%, improved on the best single client in every seed (+0.009 to +0.012 AUROC, paired bootstrap), and came within 0.004 AUROC of a model trained on all data pooled together while matching its macro F1. It outperformed every model trainable at a single site, whereas FedAvg with a shared CNN-LSTM was 0.002 AUROC better after 20 rounds and pooled-data XGBoost 0.004 better. Client-subset ablations and Shapley values showed that every institution contributes. Integrated Gradients attributed the predictions to plausible leads and waveform segments, and per-finding triage automated 53.7% of findings at a precision of 0.937 and an accuracy of 0.967 while referring the rest.

Future work. (1) Evaluate the method on genuinely heterogeneous institutions with different devices, populations and label distributions, where a learned gate is expected to matter more. (2) Add a client trained on hand-crafted amplitude features, and repeat the parameter-averaging baselines over several seeds. (3) Give the gate inputs that differ between clients, such as other modalities or site metadata, and study formal privacy guarantees such as differential privacy on the shared probabilities. (4) Improve hypertrophy recognition by restoring absolute voltage information, fit the photo triage on more validation photos, and validate the explanations with cardiologists.

CHAPTER 9

REFERENCES

[1] G. A. Roth, G. A. Mensah, C. O. Johnson *et al.*, “Global burden of cardiovascular diseases and risk factors, 1990–2019: Update from the GBD 2019 study,” *Journal of the American College of Cardiology*, vol. 76, no. 25, pp. 2982–3021, 2020.

[2] J. Schläpfer and H. J. Wellens, “Computer-interpreted electrocardiograms: Benefits and limitations,” *Journal of the American College of Cardiology*, vol. 70, no. 9, pp. 1183–1192, 2017.

[3] A. Y. Hannun, P. Rajpurkar, M. Haghpanahi, G. H. Tison, C. Bourn, M. P. Turakhia, and A. Y. Ng, “Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network,” *Nature Medicine*, vol. 25, no. 1, pp. 65–69, 2019.

[4] A. H. Ribeiro, M. H. Ribeiro, G. M. M. Paixão *et al.*, “Automatic diagnosis of the 12-lead ECG using a deep neural network,” *Nature Communications*, vol. 11, p. 1760, 2020.

[5] P. Wagner, N. Strodthoff, R.-D. Bousseljot, D. Kreiseler, F. I. Lunze, W. Samek, and T. Schaeffter, “PTB-XL, a large publicly available electrocardiography dataset,” *Scientific Data*, vol. 7, p. 154, 2020.

[6] N. Strodthoff, P. Wagner, T. Schaeffter, and W. Samek, “Deep learning for ECG analysis: Benchmarks and insights from PTB-XL,” *IEEE Journal of Biomedical and Health Informatics*, vol. 25, no. 5, pp. 1519–1528, 2021.

[7] T. Mehari and N. Strodthoff, “Self-supervised representation learning from 12-lead ECG data,” *Computers in Biology and Medicine*, vol. 141, p. 105114, 2022.

[8] Y. Na, M. Park, Y. Tae, and S. Joo, “Guiding masked representation learning to capture spatio-temporal relationship of electrocardiogram,” in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[9] T. Mehari and N. Strodthoff, “Towards quantitative precision for ECG analysis: Leveraging state space models, self-supervision and patient metadata,” *IEEE Journal of Biomedical and Health Informatics*, vol. 27, no. 11, pp. 5326–5334, 2023.

[10] J. Li, C. Liu, S. Cheng, R. Arcucci, and S. Hong, “Frozen language model helps ECG zero-shot learning,” in *Proc. Medical Imaging with Deep Learning (MIDL)*, ser. PMLR, vol. 227, 2023.

[11] C. Liu, Z. Wan, C. Ouyang, A. Shah, W. Bai, and R. Arcucci, “Zero-shot ECG classification with multimodal learning and test-time clinical knowledge enhancement,” in *Proc. International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 235, 2024.

[12] N. Rieke, J. Hancox, W. Li *et al.*, “The future of digital health with federated learning,” *npj Digital Medicine*, vol. 3, p. 119, 2020.

[13] P. Kairouz, H. B. McMahan *et al.*, “Advances and open problems in federated

learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[14] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. PMLR, vol. 54, 2017, pp. 1273–1282.

[15] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Machine Learning and Systems (MLSys)*, vol. 2, 2020, pp. 429–450.

[16] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 119, 2020, pp. 5132–5143.

[17] P. Wagner, T. Mehari, W. Haverkamp, and N. Strodthoff, "Explaining deep learning for ECG analysis: Building blocks for auditing and knowledge discovery," *Computers in Biology and Medicine*, vol. 176, p. 108525, 2024.

[18] A. R. Palakayala, P. Kuppusamy, D. Kothandaraman, G. Archana, and J. Gera, "HAMF: A novel hierarchical attention-based multi-modal fusion model for Parkinson's disease classification and severity prediction," *IEEE Access*, vol. 13, pp. 81 252–81 278, 2025.

[19] A. Raza, K. P. Tran, L. Koehl, and S. Li, "Designing ECG monitoring healthcare system with federated transfer learning and explainable AI," *Knowledge-Based Systems*, vol. 236, p. 107763, 2022.

[20] S. Goto, D. Solanki, J. E. John, R. Yagi, M. Homilius, G. Ichihara, Y. Katsumata, H. K. Gaggin, Y. Itabashi, C. A. MacRae, and R. C. Deo, "Multinational federated learning approach to train ECG and echocardiogram models for hypertrophic cardiomyopathy detection," *Circulation*, vol. 146, no. 10, pp. 755–769, 2022.

[21] C. Quan, Z. Chen, K. Ren, and Z. Luo, "FedOcw: optimized federated learning for cross-lingual speech-based Parkinson's disease detection," *npj Digital Medicine*, vol. 8, p. 357, 2025.

[22] T. A. Sathi, R. Jany, A. K. M. Azad, S. A. Alyami, N. Alotaibi, I. Hussain, and M. A. Hossain, "Explainable federated learning for multi-class heart disease diagnosis via ECG fiducial features," *Diagnostics*, vol. 15, no. 24, p. 3110, 2025.

[23] V. Jahmunah, E. Y. K. Ng, R.-S. Tan, S. L. Oh, and U. R. Acharya, "Explainable detection of myocardial infarction using deep learning models with Grad-CAM technique on ECG signals," *Computers in Biology and Medicine*, vol. 146, p. 105550, 2022.

[24] P. Pantelidis, S. Ruipérez-Campillo, J. E. Vogt, A. Antonopoulos, I. Gialamas, G. E. Zakynthinos, M. Spartalis, P. Dilaveris, J. Millet, T. G. Papaioannou, E. Oikonomou, and G. Siasos, "ECG-XPLAIM: eXPlainable locally-adaptive artificial intelligence model for arrhythmia detection from large-scale electrocardiogram data," *Frontiers in Cardiovascular Medicine*, vol. 12, p. 1659971, 2025.

[25] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[26] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[27] M. Schuster and K. K. Paliwal, “Bidirectional recurrent neural networks,” *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.

[28] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 70, 2017, pp. 1321–1330.

[29] N. Guha, A. Talwalkar, and V. Smith, “One-shot federated learning,” arXiv:1902.11175, 2019.

[30] D. Li and J. Wang, “FedMD: Heterogenous federated learning via model distillation,” arXiv:1910.03581, 2019.

[31] T. Lin, L. Kong, S. U. Stich, and M. Jaggi, “Ensemble distillation for robust model fusion in federated learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 2351–2363.

[32] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, “Adaptive mixtures of local experts,” *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991.

[33] M. Sundararajan, A. Taly, and Q. Yan, “Axiomatic attribution for deep networks,” in *Proc. International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 70, 2017, pp. 3319–3328.

[34] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 618–626.

[35] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 4765–4774.

[36] L. S. Shapley, “A value for n-person games,” in *Contributions to the Theory of Games II*, ser. Annals of Mathematics Studies. Princeton University Press, 1953, vol. 28, pp. 307–317.

[37] A. L. Goldberger, L. A. N. Amaral, L. Glass *et al.*, “PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals,” *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000.

[38] A. Paszke, S. Gross, F. Massa *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[39] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. International Conference on Learning Representations (ICLR)*, 2019.

[40] L. N. Smith and N. Topin, “Super-convergence: Very fast training of neural networks using large learning rates,” in *Proc. SPIE Artificial Intelligence and Machine Learning*

for Multi-Domain Operations Applications, vol. 11006, 2019, p. 1100612.

[41] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993.

[42] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations (ICLR)*, 2015.

[43] J. Wang, Q. Liu, H. Liang, G. Joshi, and H. V. Poor, "Tackling the objective inconsistency problem in heterogeneous federated optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 7611–7623.

[44] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 6402–6413.

[45] T. G. Dietterich, "Ensemble methods in machine learning," in *Multiple Classifier Systems (MCS)*, ser. Lecture Notes in Computer Science, vol. 1857. Springer, 2000, pp. 1–15.

APPENDIX A – Sample Code

A.1 Weighted gated aggregation (src/cardiomoe/fourclient/gate.py)

```
def mix(P: Tensor, w: Tensor) -> Tensor:
    """P [N, K clients, C], w [K] or [N, K] -> [N, C]."""
    if w.dim() == 1:
        w = w.expand(P.shape[0], -1)
    return (w.unsqueeze(-1) * P).sum(1)


def bce_prob(p: Tensor, y: Tensor) -> Tensor:
    p = p.clamp(EPS, 1 - EPS)
    return -(y * p.log() + (1 - y) * (1 - p).log()).mean()


def entropy(P: Tensor) -> Tensor:
    """Mean binary entropy per client: [N, K, C] -> [N, K]."""
    p = P.clamp(EPS, 1 - EPS)
    return -(p * p.log() + (1 - p) * (1 - p).log()).mean(-1)


class EqualWeights:
    name = "Equal-weight mean"

    def fit(self, P, y, groups=None):
        self.w = torch.full((P.shape[1],), 1.0 / P.shape[1])
        return self

    def weights(self, P: Tensor) -> Tensor:
        return self.w.expand(P.shape[0], -1)

    def predict(self, P: Tensor) -> Tensor:
        return mix(P, self.w)


class StaticWeights:
    name = "Static learned weights"

    def __init__(self, lr: float = 0.05, steps: int = 500):
        self.lr, self.steps = lr, steps

    def fit(self, P: Tensor, y: Tensor, groups=None):
        theta = torch.zeros(P.shape[1], requires_grad=True)
        opt = torch.optim.Adam([theta], lr=self.lr)
        for _ in range(self.steps):  # 4 parameters, convex-ish: full-batch
Adam converges quickly
            opt.zero_grad()
            loss = bce_prob(mix(P, torch.softmax(theta, 0)), y)
            loss.backward()
            opt.step()
        self.w = torch.softmax(theta.detach(), 0)
        return self

    def weights(self, P: Tensor) -> Tensor:
        return self.w.expand(P.shape[0], -1)

    def predict(self, P: Tensor) -> Tensor:
        return mix(P, self.w)
```

A.2 Client D and the client wrapper (src/cardiomoe/fourclient/models.py)

```
class CNNLSTM(nn.Module):
    def __init__(self, channels=(32, 64, 128), hidden: int = 128):
        super().__init__()
        layers, c = [], 12
        for ch in channels:
            layers.append(conv_block(c, ch))
            c = ch
        self.conv = nn.Sequential(*layers)  # 1000 -> 125 time steps
        self.lstm = nn.LSTM(c, hidden, batch_first=True, bidirectional=True)
        self.out_dim = 4 * hidden

    def forward(self, x: Tensor) -> Tensor:
        h, _ = self.lstm(self.conv(x).transpose(1, 2))
        return avg_max(h, 1)


ARCHS = {"cnn1d": CNN1D, "resnet1d": ResNet1D, "bilstm": BiLSTM, "cnn_lstm":
CNNLSTM}
ARCH_NAMES = {"cnn1d": "1D CNN", "resnet1d": "1D ResNet", "bilstm": "BiLSTM",
"cnn_lstm": "CNN-LSTM"}



class SignalClient(BaseClient):
    modality = "ecg"

    def __init__(self, arch: str, embed_dim: int = 256, dropout: float = 0.3,
n_classes: int = 5):
        super().__init__()
        self.arch, self.embed_dim = arch, embed_dim
        self.backbone = ARCHS[arch]()
        self.proj = nn.Sequential(nn.Linear(self.backbone.out_dim, embed_dim),
nn.ReLU(), nn.Dropout(dropout))
        self.head = nn.Linear(embed_dim, n_classes)

    def encode(self, batch: dict) -> Tensor:
        return self.proj(self.backbone(batch["signal"]))
```

A.3 Temperature scaling (src/cardiomoe/fusion/calibration.py)

```
def fit_temperature(logits: torch.Tensor, y: torch.Tensor, max_iter: int = 200)
-> float:
    logits, y = logits.float().detach(), y.float()
    log_t = torch.zeros((), requires_grad=True)
    opt = torch.optim.LBFGS([log_t], lr=0.1, max_iter=max_iter,
line_search_fn="strong_wolfe")

    def closure():
        opt.zero_grad()
        loss = masked_bce(logits / log_t.exp(), y)
        loss.backward()
        return loss

    opt.step(closure)
    return float(log_t.detach().exp().clamp(0.05, 20.0))
```

A.4 Per-finding triage rule (src/cardiomoe/fourclient/triage.py)

```
class Triage:
    def __init__(self, bands: dict):
        self.bands = bands

    def decide(self, p: np.ndarray) -> np.ndarray:
```

```
        """[N, 5] probabilities -> [N, 5] in {1, 0, -1 (indeterminate)}."""
        out = np.full(p.shape, INDETERMINATE, dtype=int)
        for i, c in enumerate(SUPERCLASSES):
            b = self.bands[c]
            if b["lo"] is not None:
                out[p[:, i] <= b["lo"], i] = 0
            if b["hi"] is not None:
                out[p[:, i] >= b["hi"], i] = 1
        return out

    @classmethod
    def from_dict(cls, d: dict) -> "Triage":
        return cls(d["bands"])
```